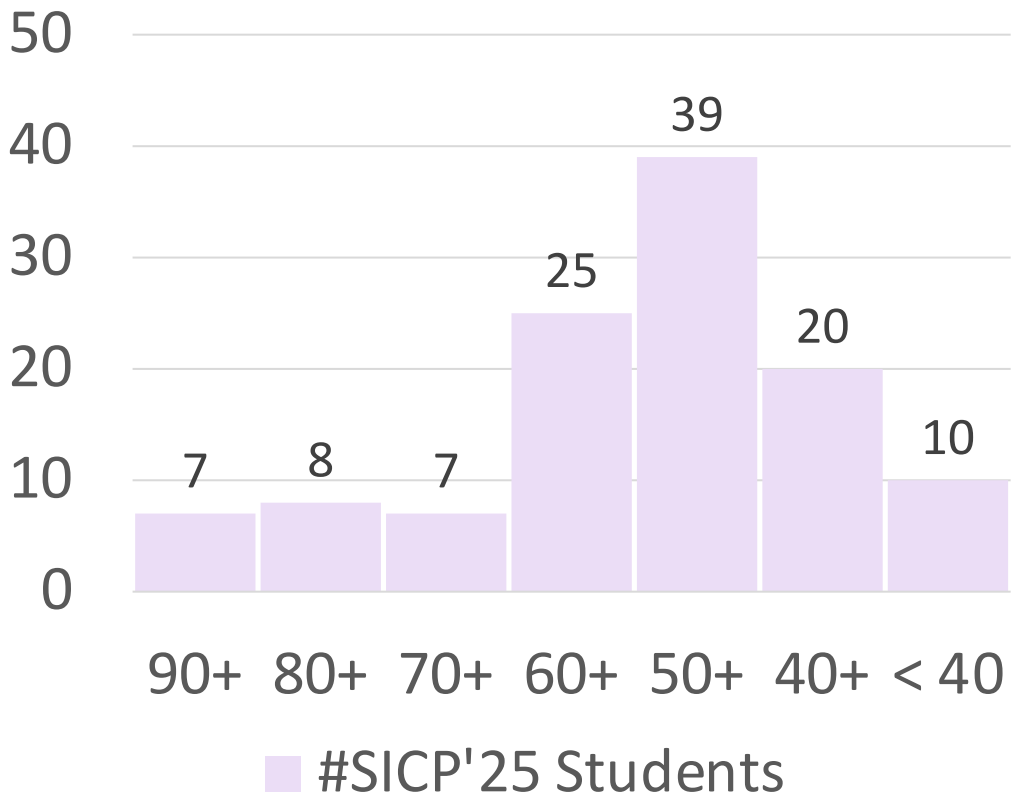
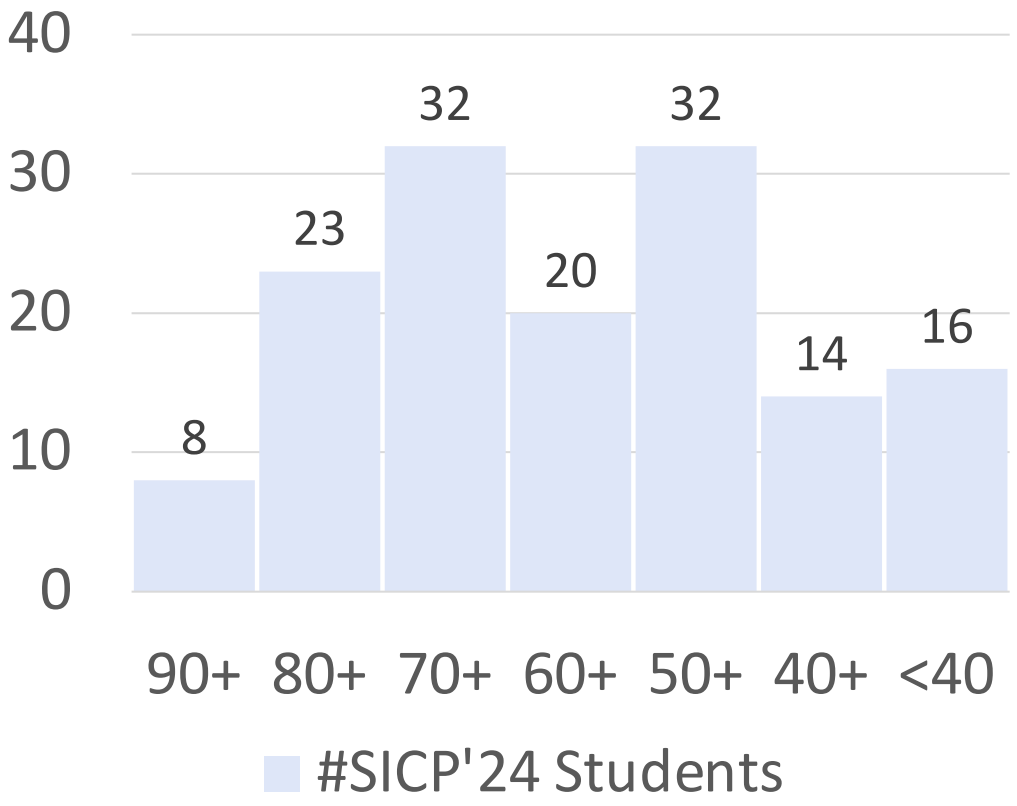
A TA's Reflection



We’ve evolved 4.84pts more evil this year …

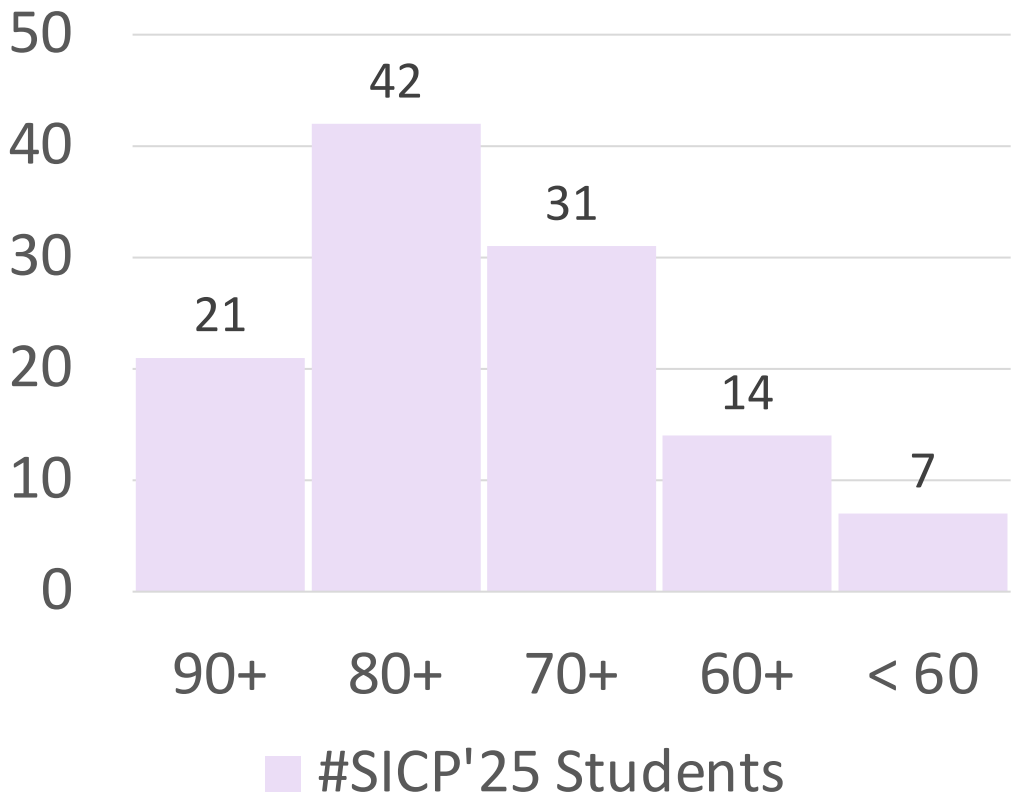


Maximum: 99 / **Average: 58.97** / Median: 57.5

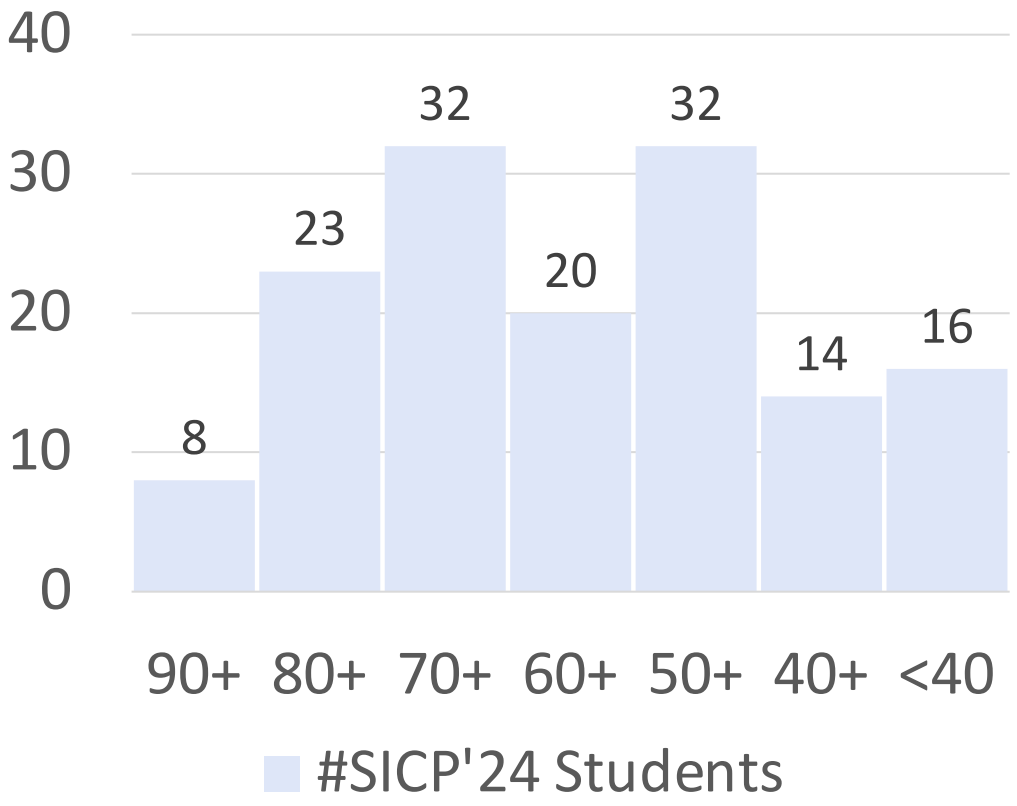


Maximum: 99 / **Average: 63.61** / Median: 66

We've evolved 4.84pts more evil this year ...

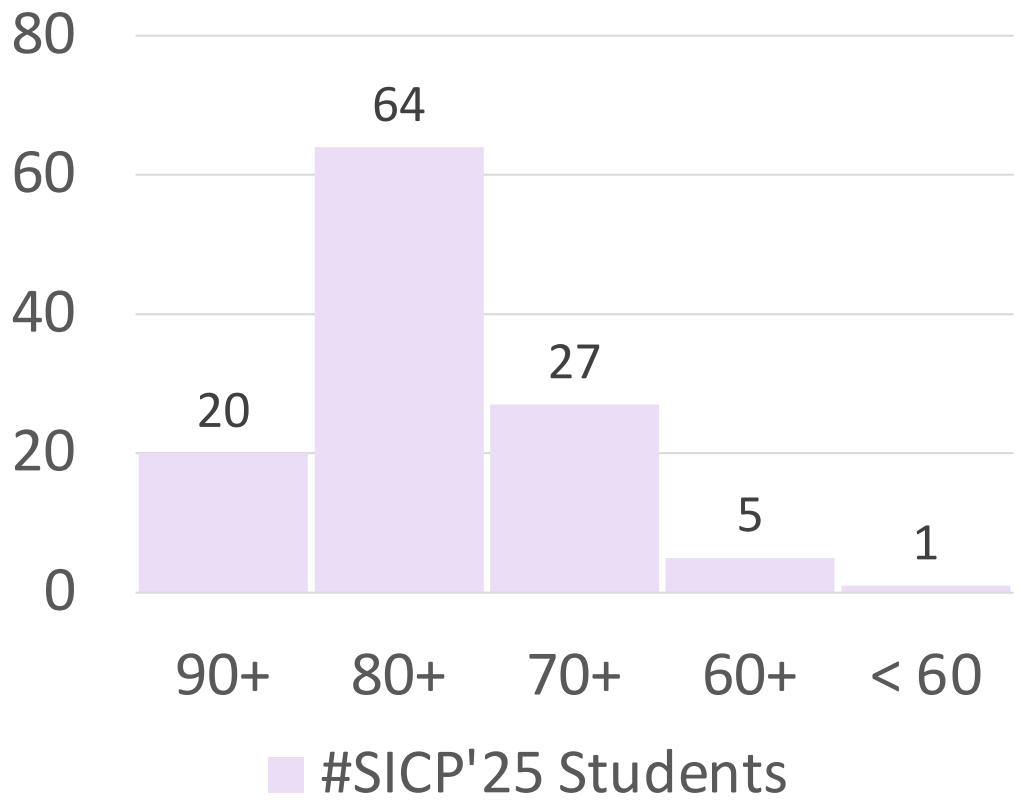


Maximum: 99 / **Average: 79.57** / Median: 80.5

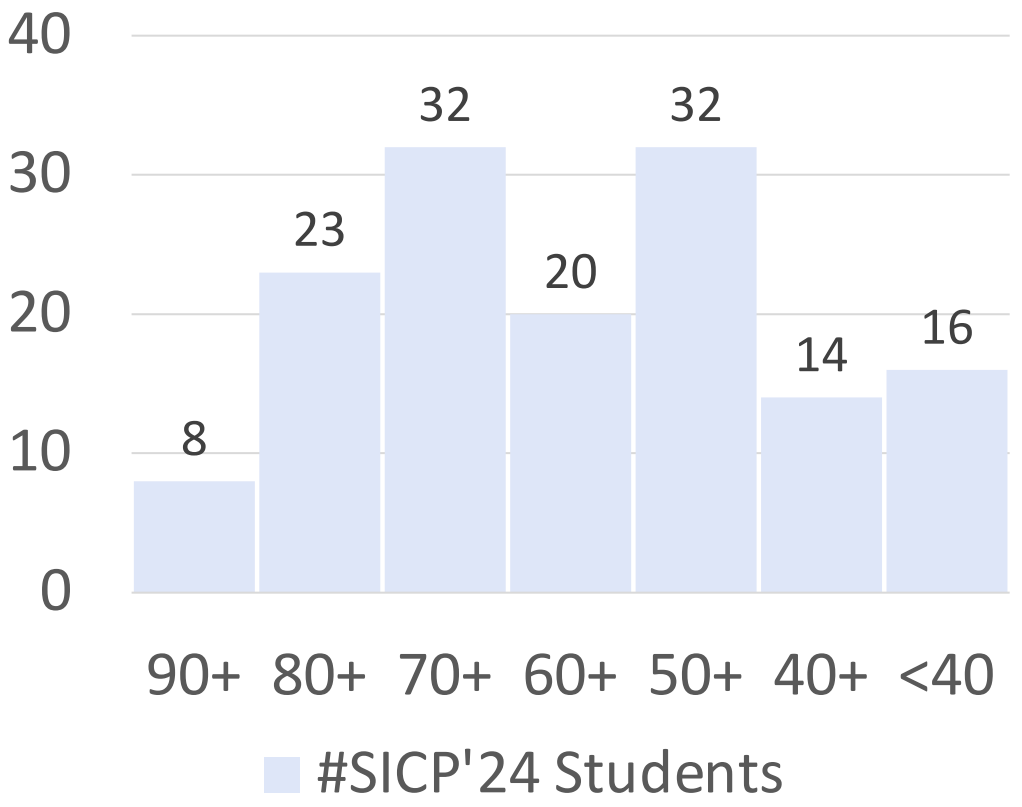


Maximum: 99 / **Average: 63.61** / Median: 66

We've evolved 4.84pts more evil this year ...



Maximum: 99.3 / **Average: 82.9** / Median: 84



Maximum: 99 / **Average: 63.61** / Median: 66

Somehow, ...

	P1	P2	P3	P4	P5	P6	P7	Bonus
Average	8.88	6.28	5.60	5.04	11.87	8.11	11.87	1.11
Total	15	10	9	12	19	18	17	3
Rate	59.20%	62.8%	62.2%	42%	62.47%	45.06%	69.82%	37.00%

Somehow, I unexpectedly seem ...

	P1	P2	P3	P4	P5	P6	P7	Bonus
Average	8.88	6.28	5.60	**5.04**	11.87	8.11	11.87	1.11
Total	15	10	9	**12**	19	18	17	3
Rate	59.20%	62.8%	62.2%	**42%**	62.47%	45.06%	69.82%	37.00%

One Function for All ?

助教-崔家才 2025/10/21 18:15 置顶

sicp.pascal-lab.net/2 25/

NJU Practical Resourse Course tmp BearSir PASCAL java verilog chisel ChiselPlayers

Structure and Interpretation of Computer Programs

Fall 2025, Nanjing University

Course Information

Class Hours: 星期三 3-4节 与 星期五 5-6节， 教120

Lab Hours: 星期五 7-8节，南园综合楼502、503

Instructors: Xinyu Feng (冯新宇) Yue Li (李樾)

TAs: Jiacai Cui (崔家才) Yinfeng Lin (林吟风) Jinpeng Wang (王锦朋) Nairen Zhang (张乃仁) Hengbin Zheng (郑恒彬)

Textbooks and References:

- Textbooks: Composing Programs
- Courseware (lecture notes, homework and projects) taken from Berkeley CS 61A
- References: Harold Abelson, Gerald Jay Sussman and Julie Sussman, Structure and Interpretation of Computer Programs

One Function for All !

Jiacai Cui

PASCAL Group @ Nanjing University

Recap: When we met higher-order functions

```
def sum_naturals(n):
    """Sum the first N
    natural numbers."""
    total, k = 0, 1
    while k <= n:
        total += k
        k += 1
    return total
```

```
def sum_cubes(n):
    """Sum the first N
    cubes."""
    total, k = 0, 1
    while k <= n:
        total += pow(k, 3)
        k += 1
return total
```

Recap: When we met higher-order functions

```
def sum_naturals(n):
    """Sum the first N
    natural numbers."""
    total, k = 0, 1
    while k <= n:
        total += k
        k += 1
    return total
```

```
def sum_cubes(n):
    """Sum the first N
    cubes."""
    total, k = 0, 1
    while k <= n:
        total += pow(k, 3)
        k += 1
return total
```

Generalization

```
def summation(n, term):
    """Sum the first N
    terms."""
    total, k = 0, 1
    while k <= n:
        total += term(k)
        k += 1
    return total
```

Recap: When we met higher-order functions

. . . Generalization

```
def summation(n, term):
    """Sum the first N
    terms."""
    total, k = 0, 1
    while k <= n:
        total += term(k)
        k += 1
    return total
```

```
sum(n, lambda x: x)
"""Sum the first N
natural numbers."""
```

```
sum(n, lambda x: x * x)
"""Sum the first N
squares."""
```

```
sum(n, lambda x: pow(x, 3))
"""Sum the first N
cubes."""
```

```
sum(n, lambda x: ???)
"""Sum the first N
???s."""
```

. . . . . .

Recap: When we met higher-order functions

One Function for All !

. . . Generalization

```
def summation(n, term):
    """Sum the first N
    terms."""
```



Specialization

```
sum(n, Lambda x: x)
"""Sum the first N
natural numbers."""
```

```
sum(n, Lambda x: x * x)
"""Sum the first N
squares."""
```

```
sum(n, Lambda x: pow(x, 3))
"""Sum the first N
cubes."""
```

```
sum(n, Lambda x: ???)
"""Sum the first N
???s."""
```

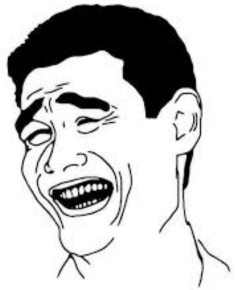
. . . . . .

Powerful Abstraction Again!

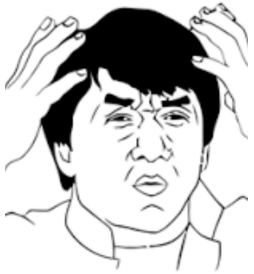
(Happy Program by Specification)

One Function for All !

- Lots of Functions (4 pts)
- Maybe Not So Many Functions ... (6 pts)
- Just One Function! (2 pts)







Lots of Functions

Old Friends for Lists

`map(f, lst)`

`filter(p, lst)`

`reversed(lst)`

`foldl(f, z, lst)`

(2 pts)

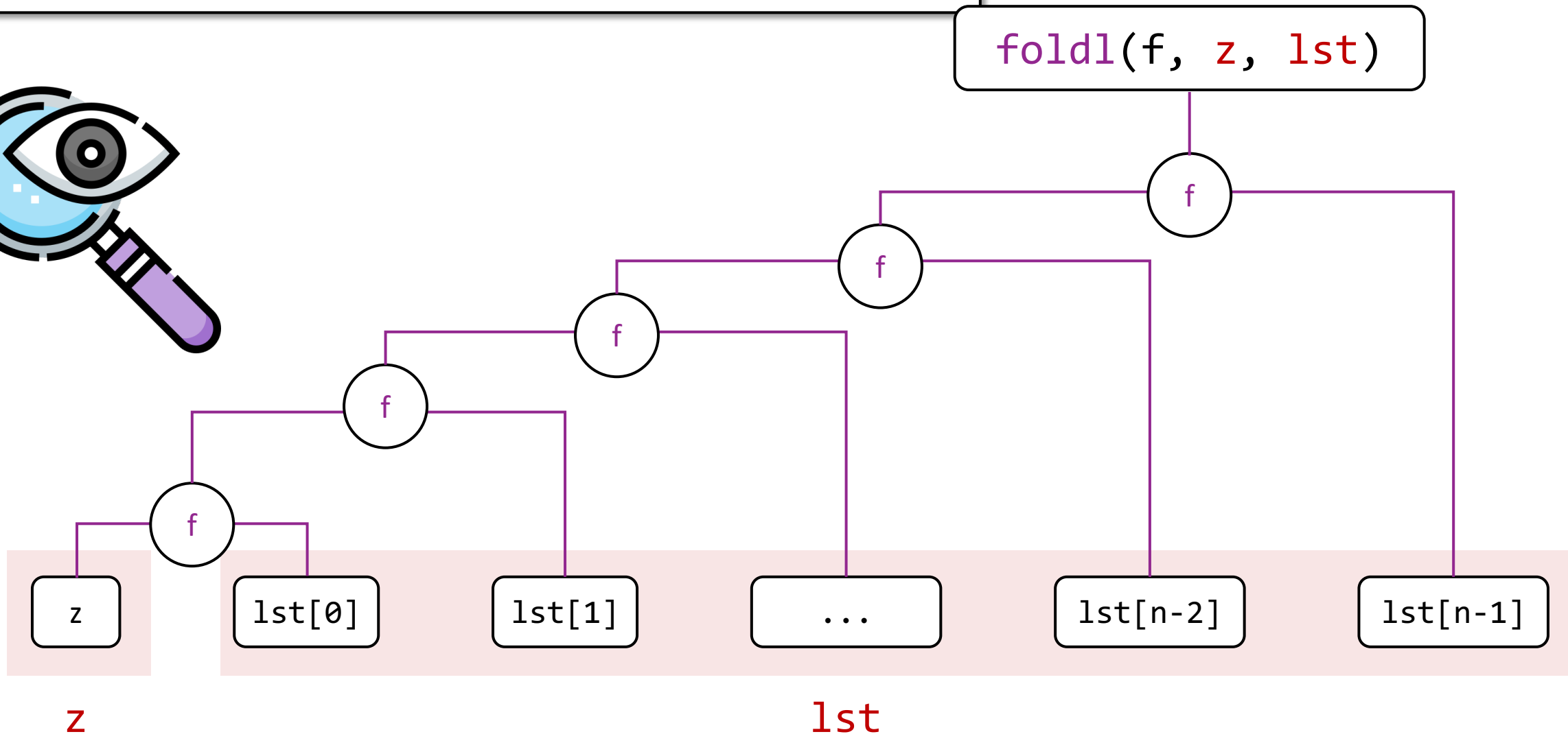
`foldr(f, z, lst)`

(2 pts)

New Friends for Lists

```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
    return foldl(f, _____________, _______) if lst else z
```

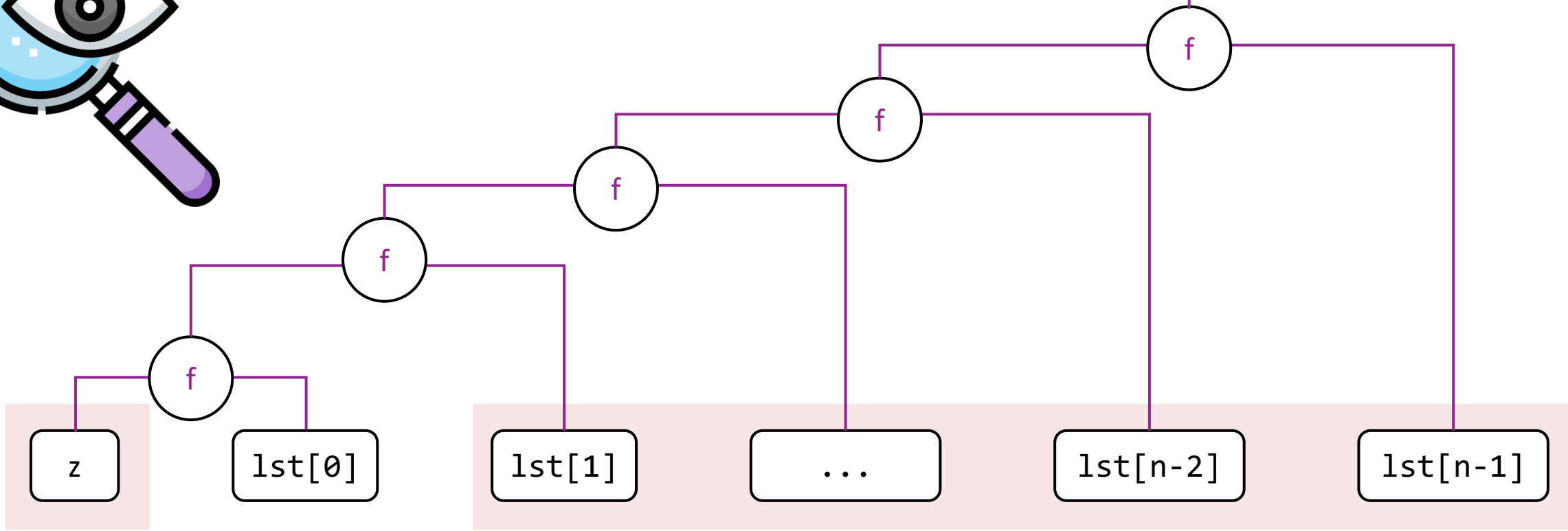




```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
    return foldl(f, _____________, lst[1:]) if lst else z
```



foldl(f, ???, lst[1:])



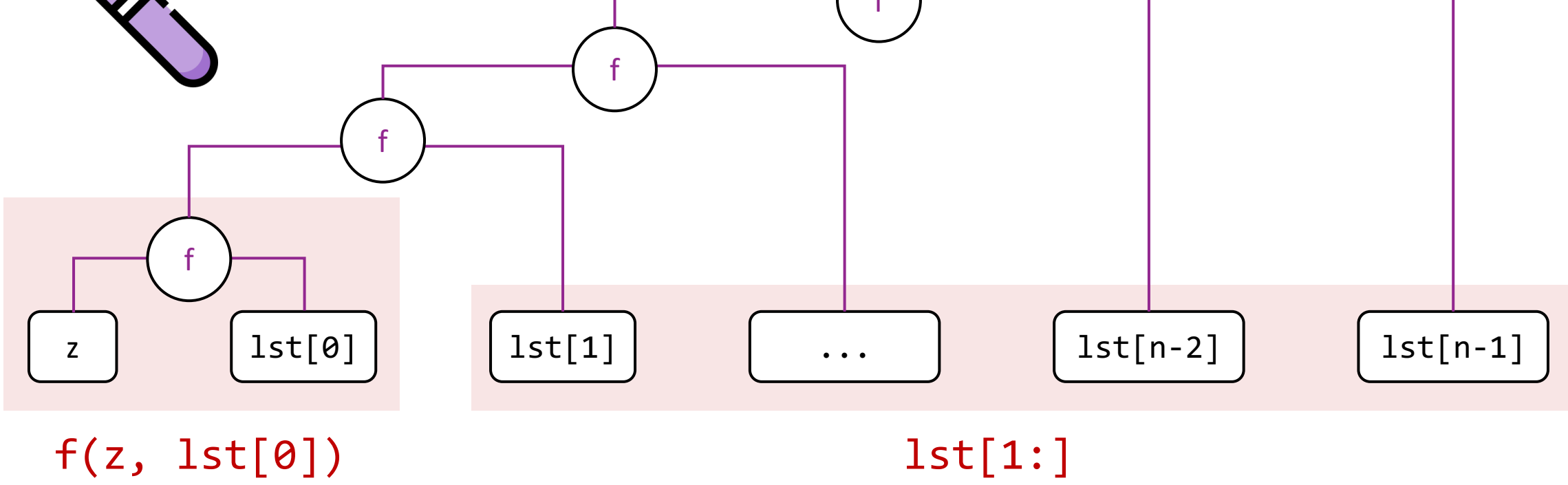
z

lst[1:]

```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
    return foldl(f, f(z, lst[0]), lst[1:]) if lst else z
```

foldl(f, f(z, lst[0]), lst[1:])





Lots of Functions

Old Friends for Lists

```
map(f, lst)
```

```
filter(p, lst)
```

```
reversed(lst)
```

```
foldl(f, z, lst)
```

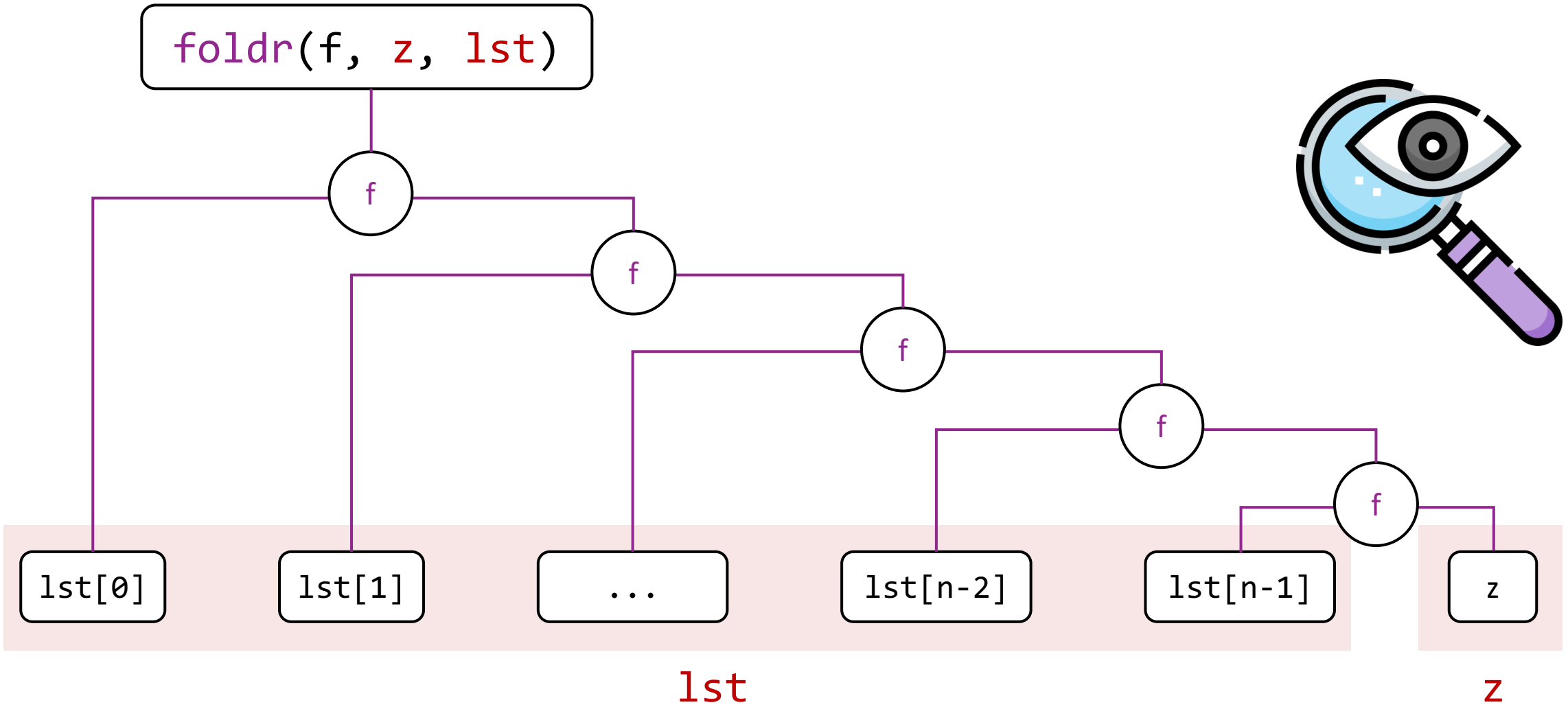
(2 pts)

```
foldr(f, z, lst)
```

(2 pts)

New Friends for Lists

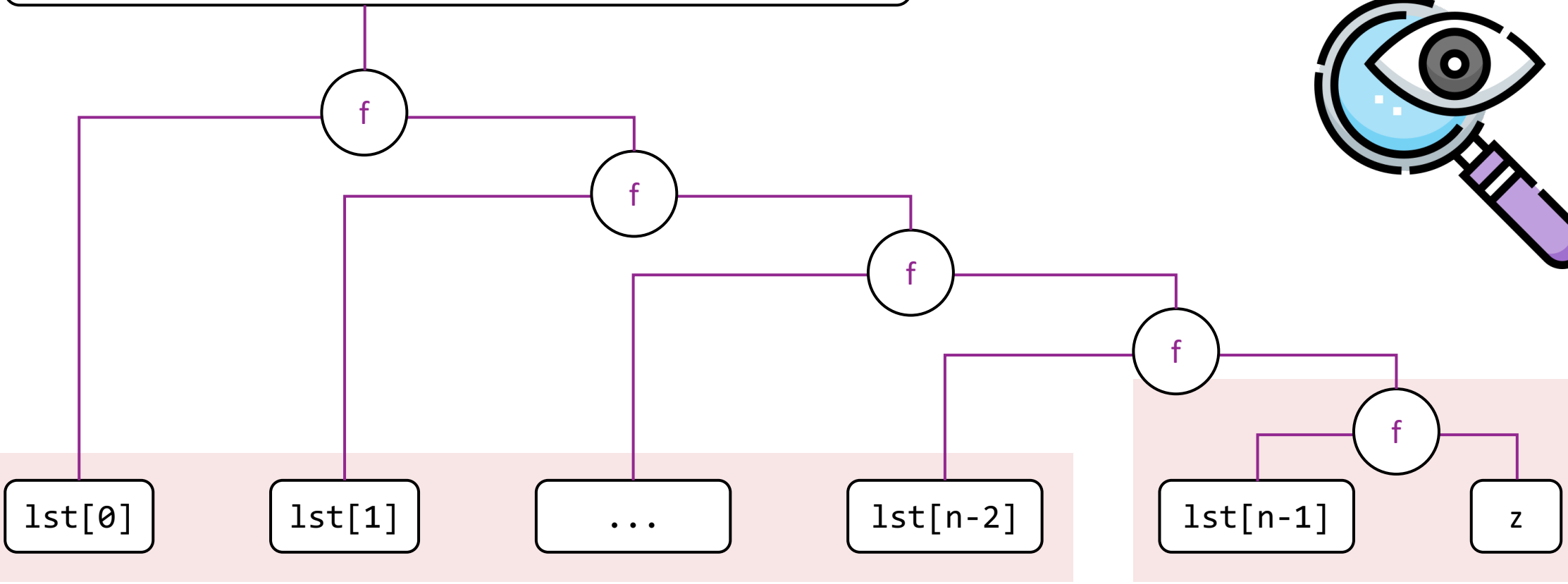
```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
    return f(_______, ____________________) if lst else z
```



```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
    return f(_______, ____________________) if lst else z
```

`foldr(f, f(lst[n-1], z), lst[:n-1])`

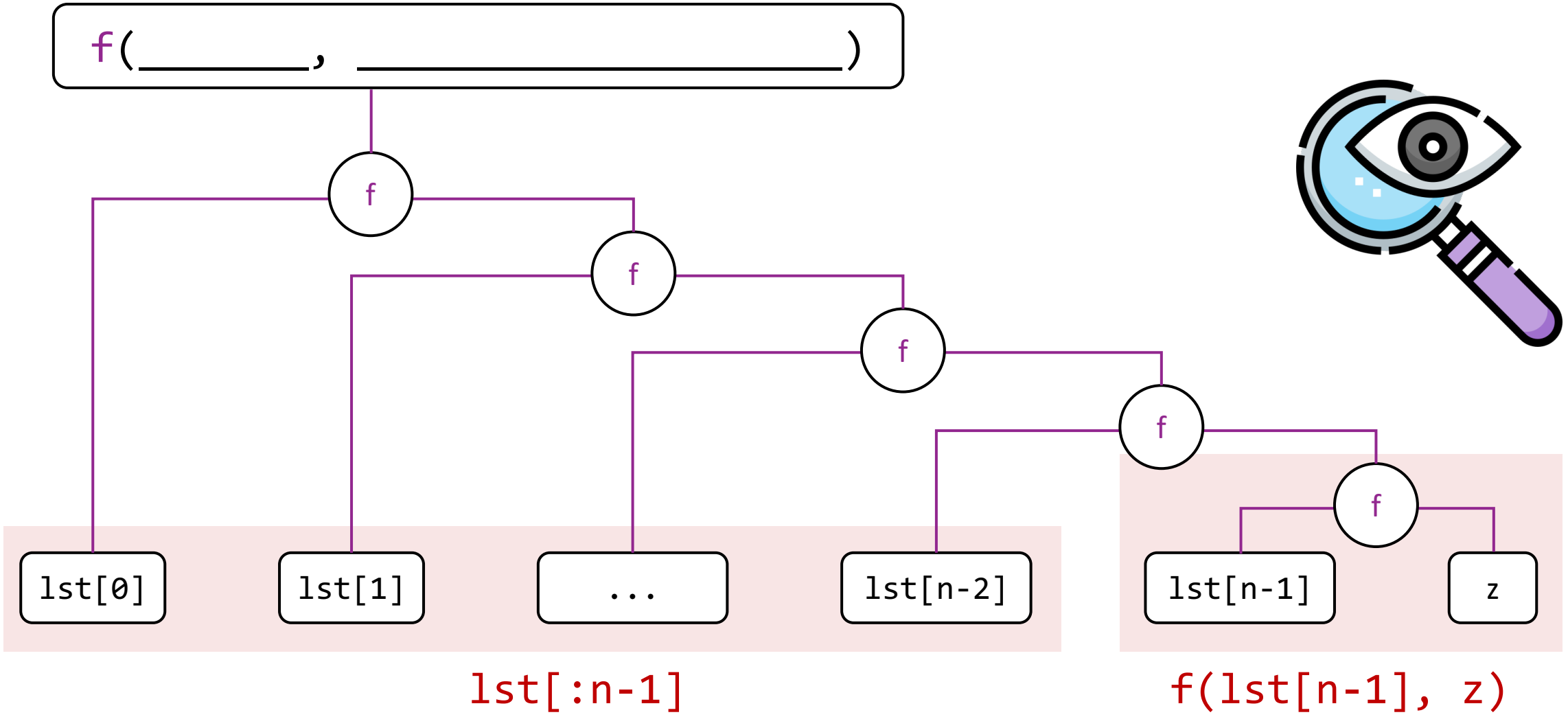




`lst[:n-1]`

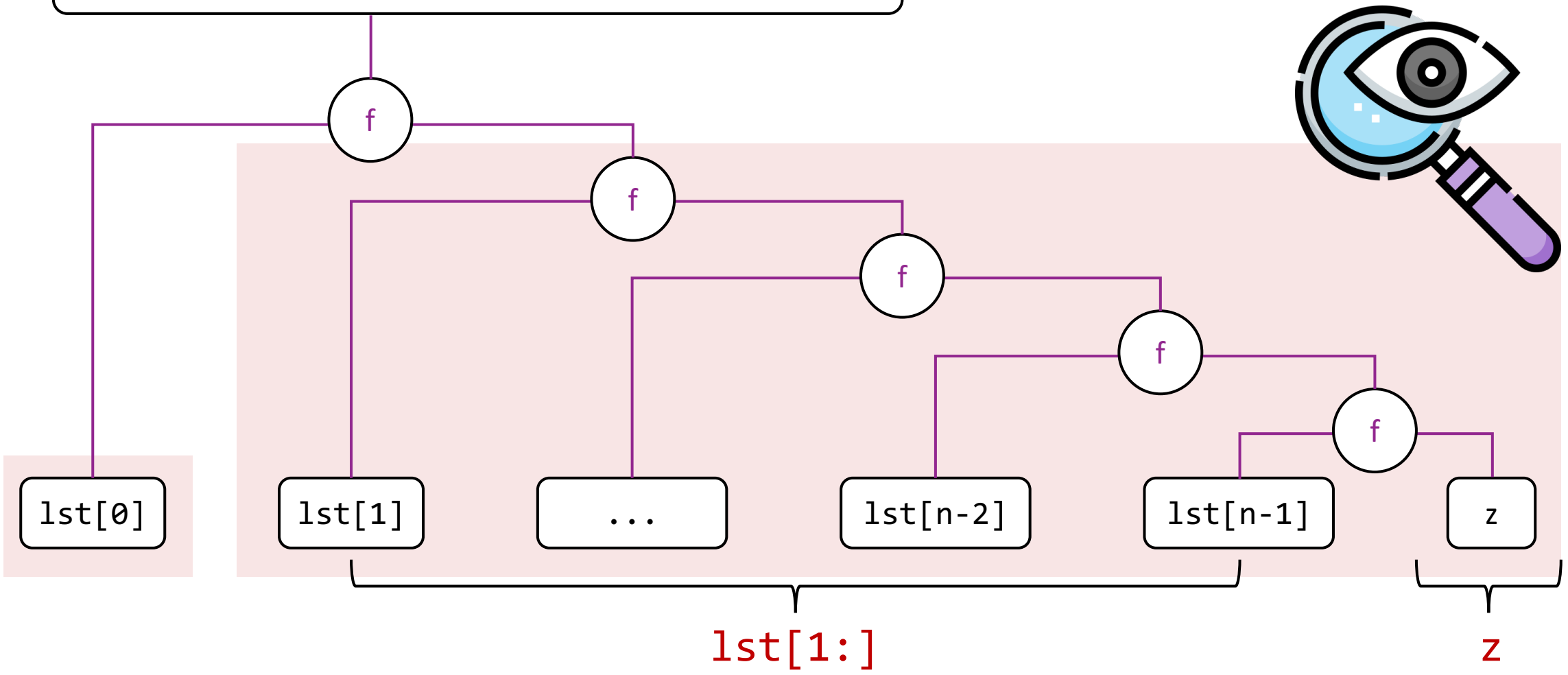
`f(lst[n-1], z)`

```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
    return f(_____, _____________________) if lst else z
```



```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
    return f(lst[0], foldr(f, z, lst[1:])) if lst else z
```

f(lst[0], foldr(f, z, lst[1:]))



Lots of Functions

Old Friends for Lists

```
map(f, lst)
```

```
filter(p, lst)
```

```
reversed(lst)
```

```
foldl(f, z, lst)
```

(2 pts)

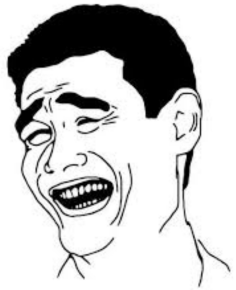
```
foldr(f, z, lst)
```

(2 pts)

New Friends for Lists

One Function for All !

- Lots of Functions (4 pts)

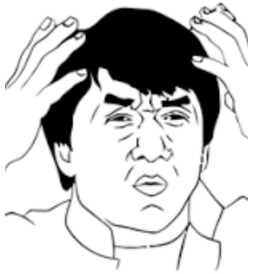


- Maybe Not So Many Functions ... (6 pts)

8 - 10pts
Expected



- Just One Function! (2 pts)



Maybe Not So Many Functions ...

Reimplement Old Friends for Lists

`map(f, lst)` (2 pts)

`filter(p, lst)` (2 pts)

`reversed(lst)` (2 pts)

```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
```

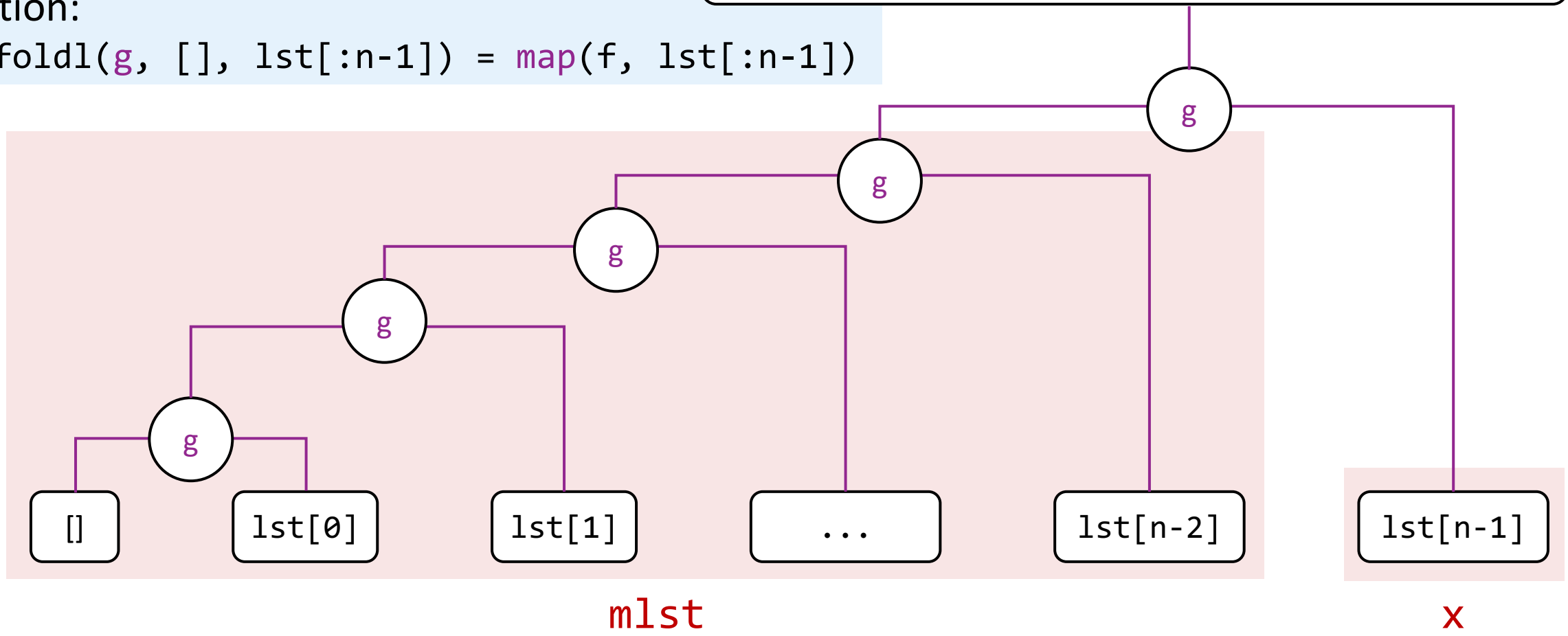
```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
```

```
def map(f, lst):
    """Returns [f(lst[0]), f(lst[1]), ..., f(lst[n-1])]."""
    return foldl(_____g_____, [], lst)
```

```
g = lambda mlst, x: ???
```

Observation:

```
mlst = foldl(g, [], lst[:n-1]) = map(f, lst[:n-1])
```

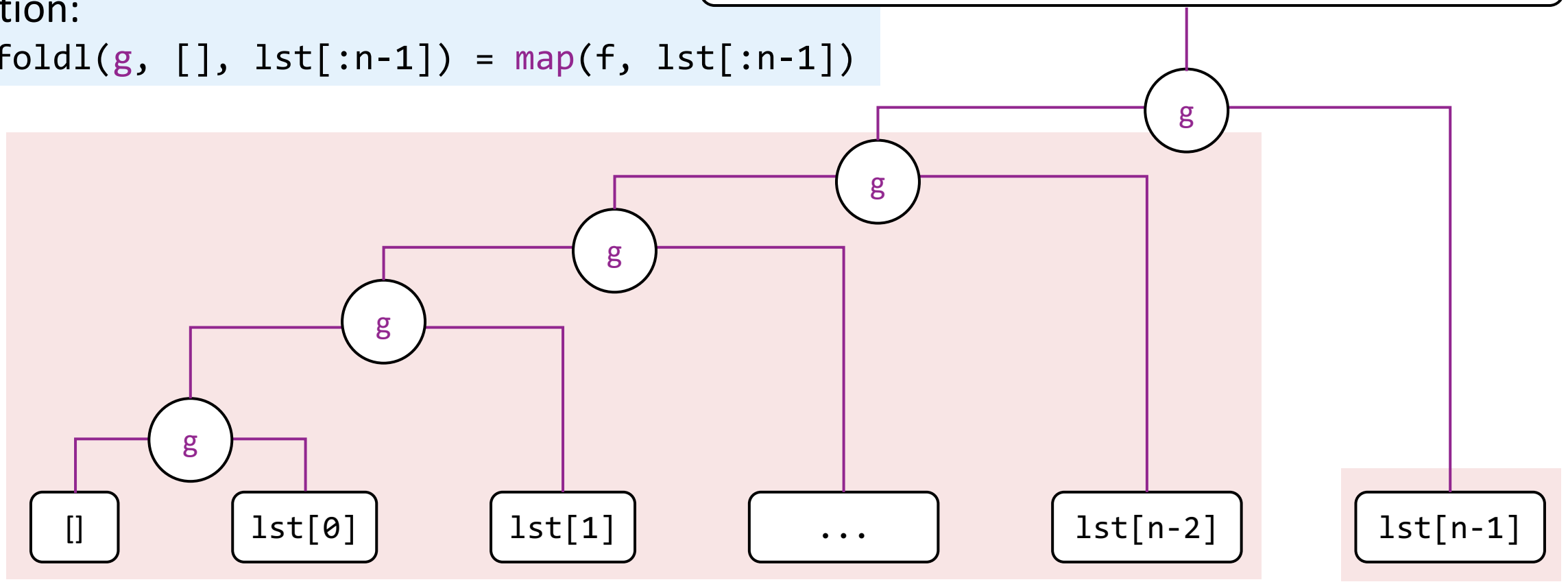


```
def map(f, lst):
    """Returns [f(lst[0]), f(lst[1]), ..., f(lst[n-1])]."""
    return foldl(_____g_____, [], lst)
```

```
g = lambda mlst, x: ???
```

Observation:

```
mlst = foldl(g, [], lst[:n-1]) = map(f, lst[:n-1])
```



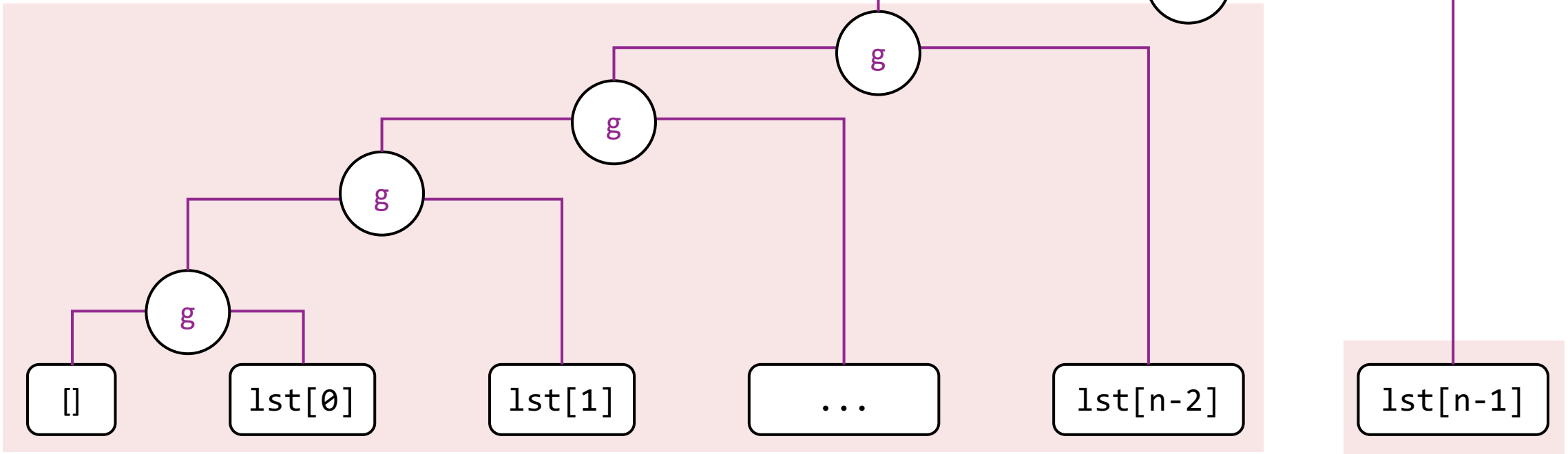

```
mlst = [f(lst[0]), f(lst[1]), ..., f(lst[n-2])]    x = lst[n-1]
```

```
def map(f, lst):
    """Returns [f(lst[0]), f(lst[1]), ..., f(lst[n-1])]."""
    return foldl(_____g_____, [], lst)
```

```
g = lambda mlst, x: mlst + [f(x)]
```

Observation:

```
mlst = foldl(g, [], lst[:n-1]) = map(f, lst[:n-1])
```




```
mlst = [f(lst[0]), f(lst[1]), ..., f(lst[n-2])]    x = lst[n-1]
```

Maybe Not So Many Functions ...

Reimplement Old Friends for Lists

`map(f, lst)` (2 pts)

`filter(p, lst)` (2 pts)

`reversed(lst)` (2 pts)

```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
```

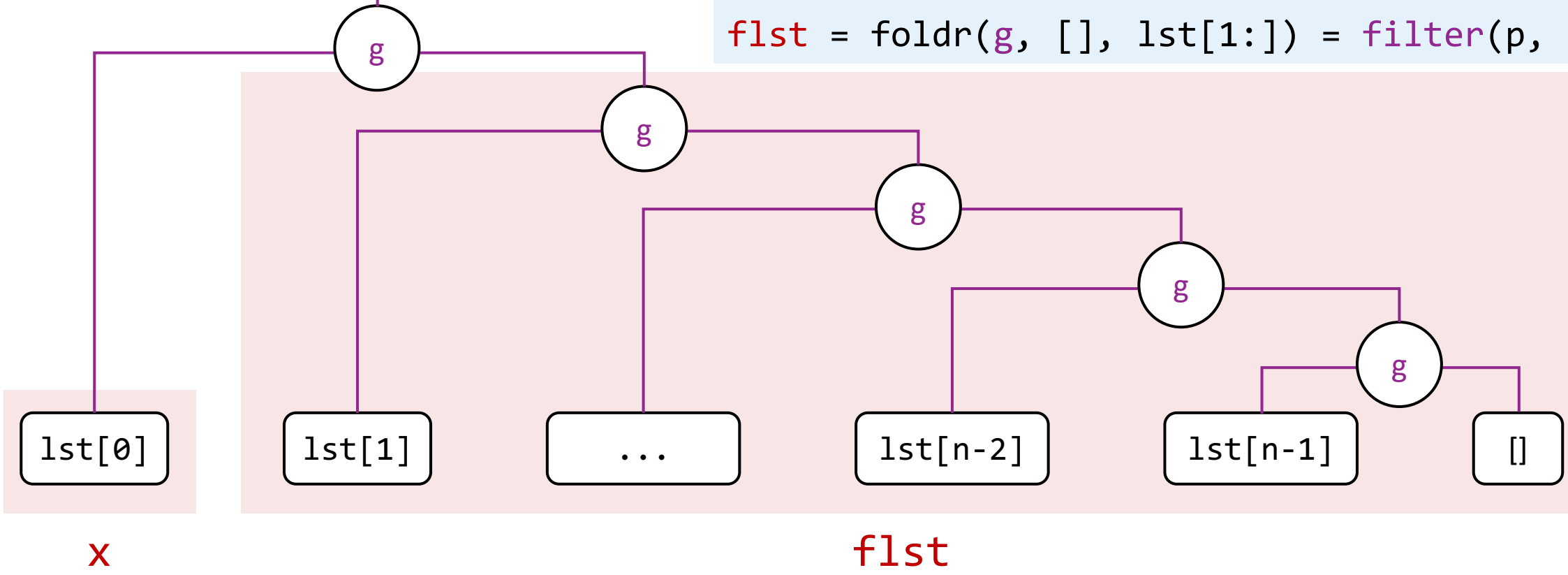
```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
```

```
def filter(p, lst):
    """Returns [x in lst if p(x)]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, flst: ???
```

Observation:

`flst = foldr(g, [], lst[1:]) = filter(p, lst[1:])`

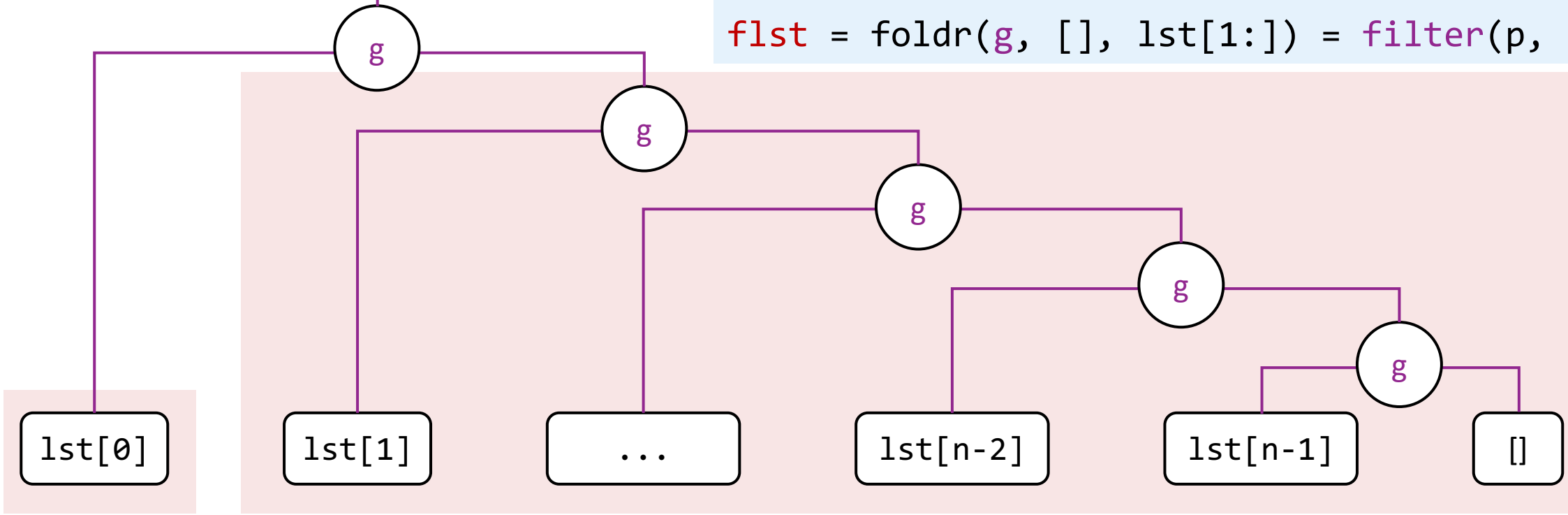


```
def filter(p, lst):
    """Returns [x in lst if p(x)]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, flst: ???
```

Observation:

`flst = foldr(g, [], lst[1:]) = filter(p, lst[1:])`



`x = lst[0]`

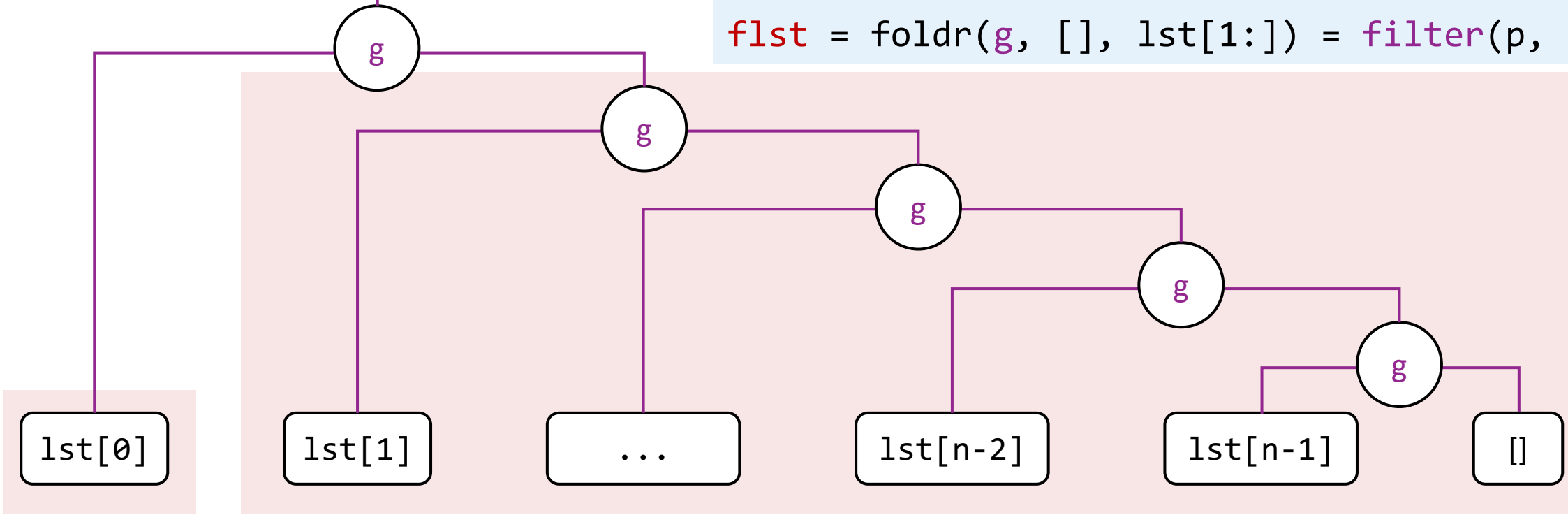
`flst = [x in lst[1:] if p(x)]`

```
def filter(p, lst):
    """Returns [x in lst if p(x)]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, flst:     ???     if p(x) else  ???
```

Observation:

`flst = foldr(g, [], lst[1:]) = filter(p, lst[1:])`



`x = lst[0]`

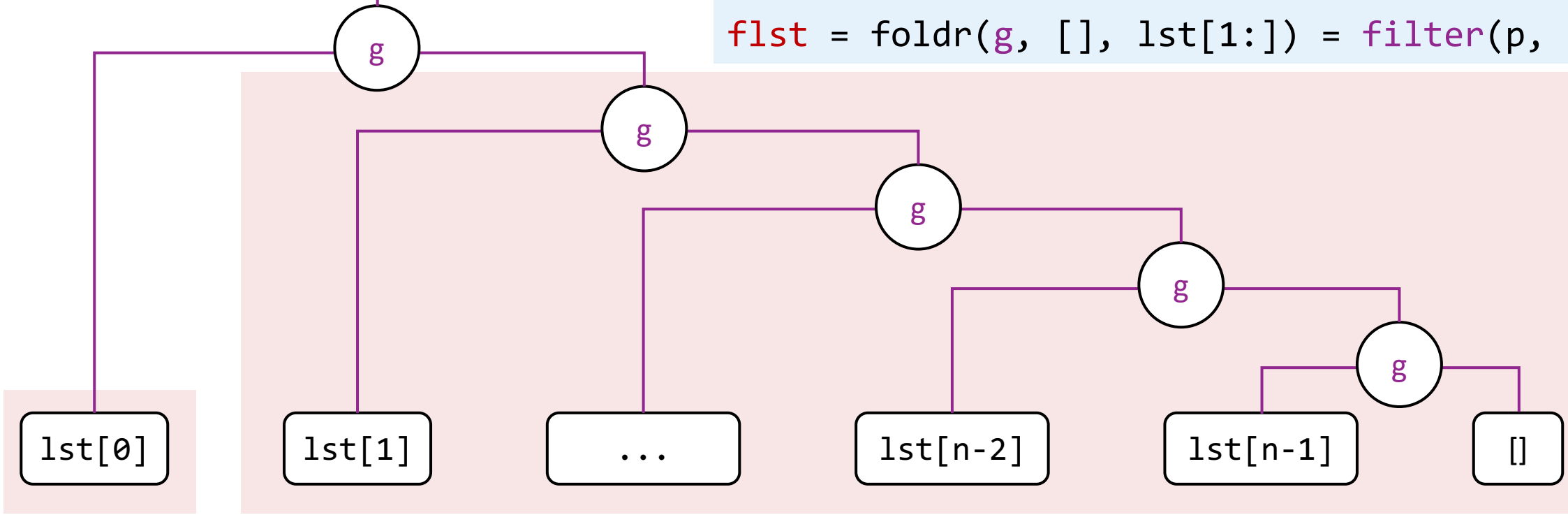
`flst = [x in lst[1:] if p(x)]`

```
def filter(p, lst):
    """Returns [x in lst if p(x)]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, flst:     ???     if p(x) else flst
```

Observation:

`flst = foldr(g, [], lst[1:]) = filter(p, lst[1:])`



`x = lst[0]`

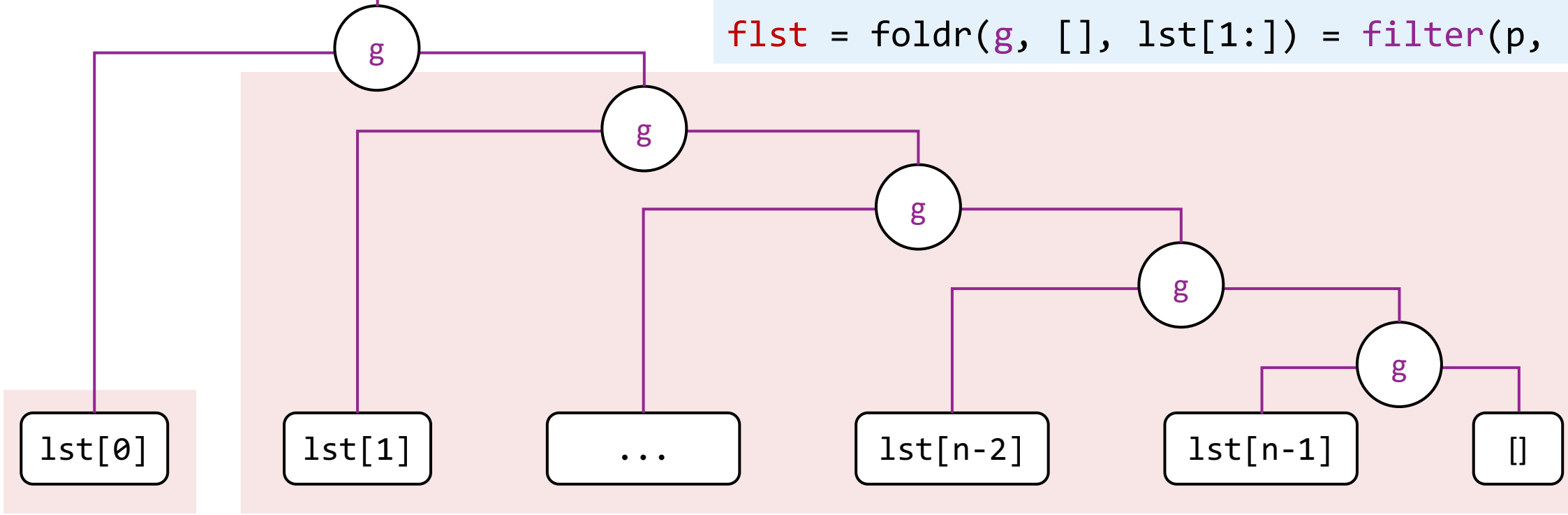
`flst = [x in lst[1:] if p(x)]`

```
def filter(p, lst):
    """Returns [x in lst if p(x)]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, flst: [x] + flst if p(x) else flst
```

Observation:

`flst = foldr(g, [], lst[1:]) = filter(p, lst[1:])`



`x = lst[0]`

`flst = [x in lst[1:] if p(x)]`

Maybe Not So Many Functions ...

Reimplement Old Friends for Lists

`map(f, lst)` (2 pts)

`filter(p, lst)` (2 pts)

`reversed(lst)` (2 pts)

```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
```

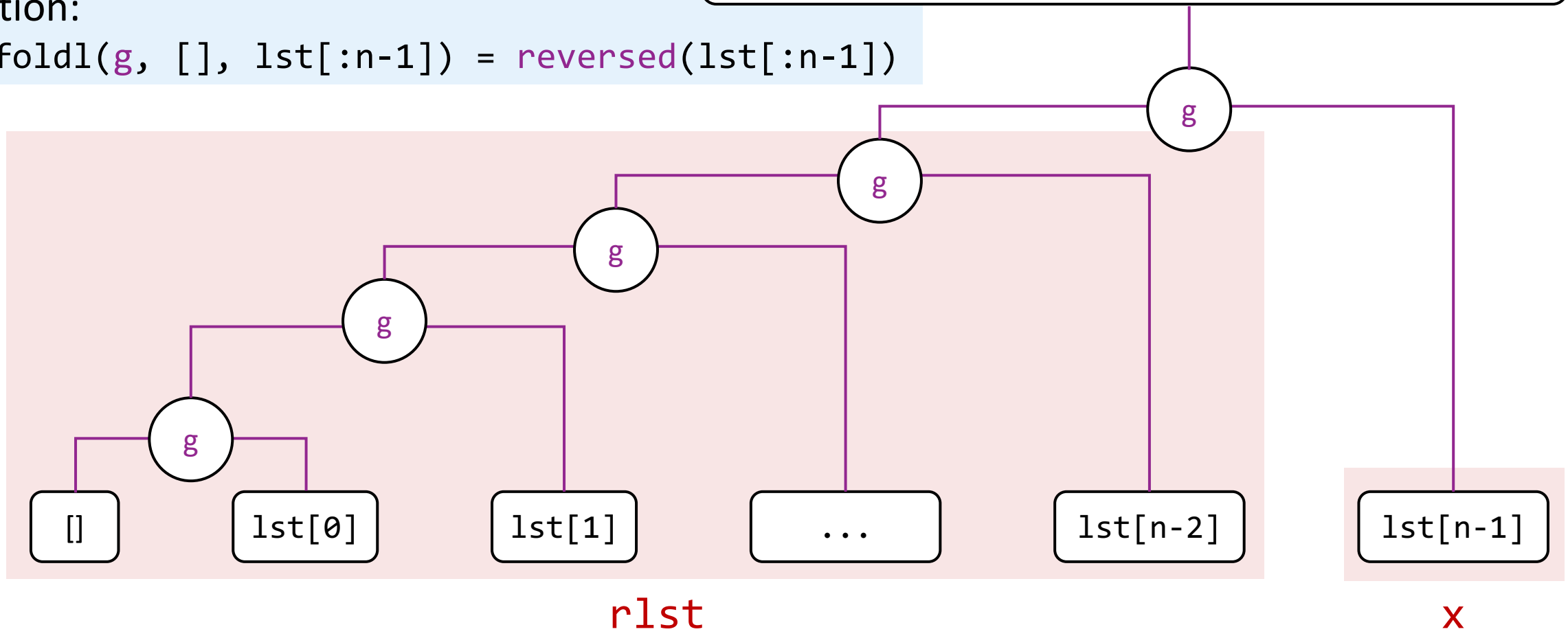
```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
```

```
def reversed(lst):
    """Returns [lst[n-1], lst[n-2], ..., lst[0]]."""
    return foldl(_____g_____, [], lst)
```

```
g = lambda rlst, x: ???
```

Observation:

`rlst = foldl(g, [], lst[:n-1]) = reversed(lst[:n-1])`

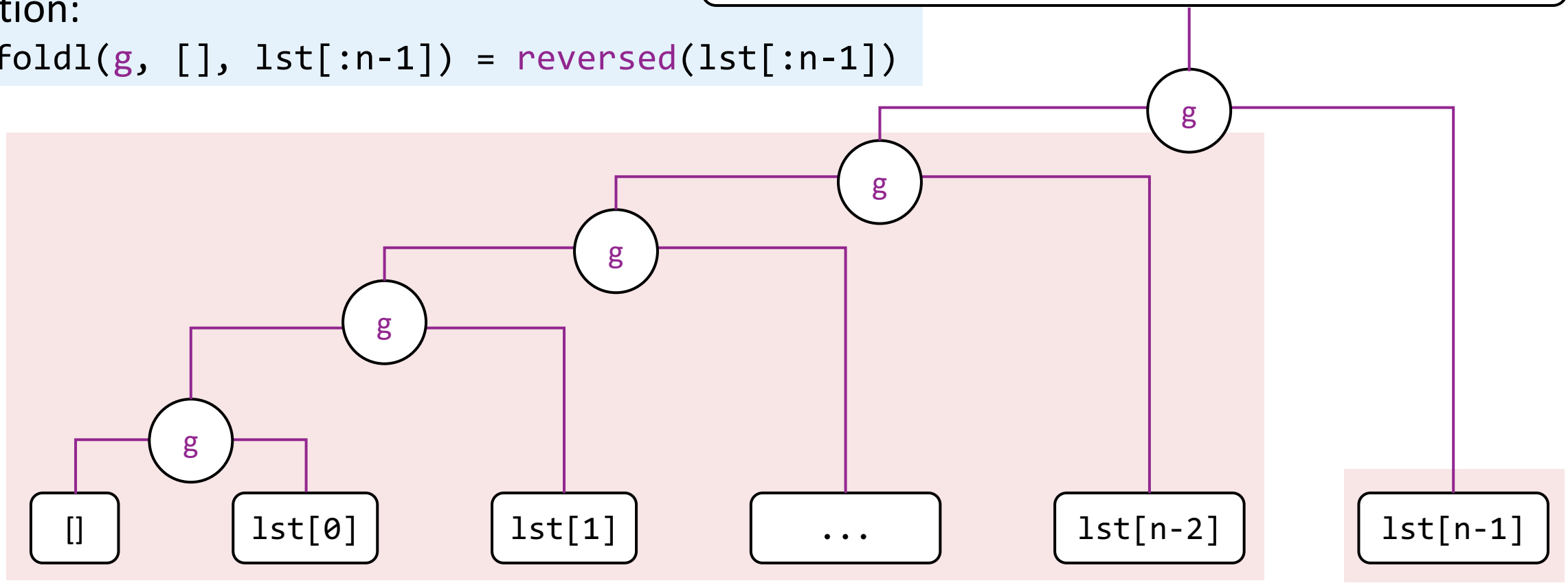


```
def reversed(lst):
    """Returns [lst[n-1], lst[n-2], ..., lst[0]]."""
    return foldl(_____g_____, [], lst)
```

```
g = lambda rlst, x: ???
```

Observation:

`rlst = foldl(g, [], lst[:n-1]) = reversed(lst[:n-1])`



`rlst = [lst[n-2], lst[n-3], ..., lst[0]]`

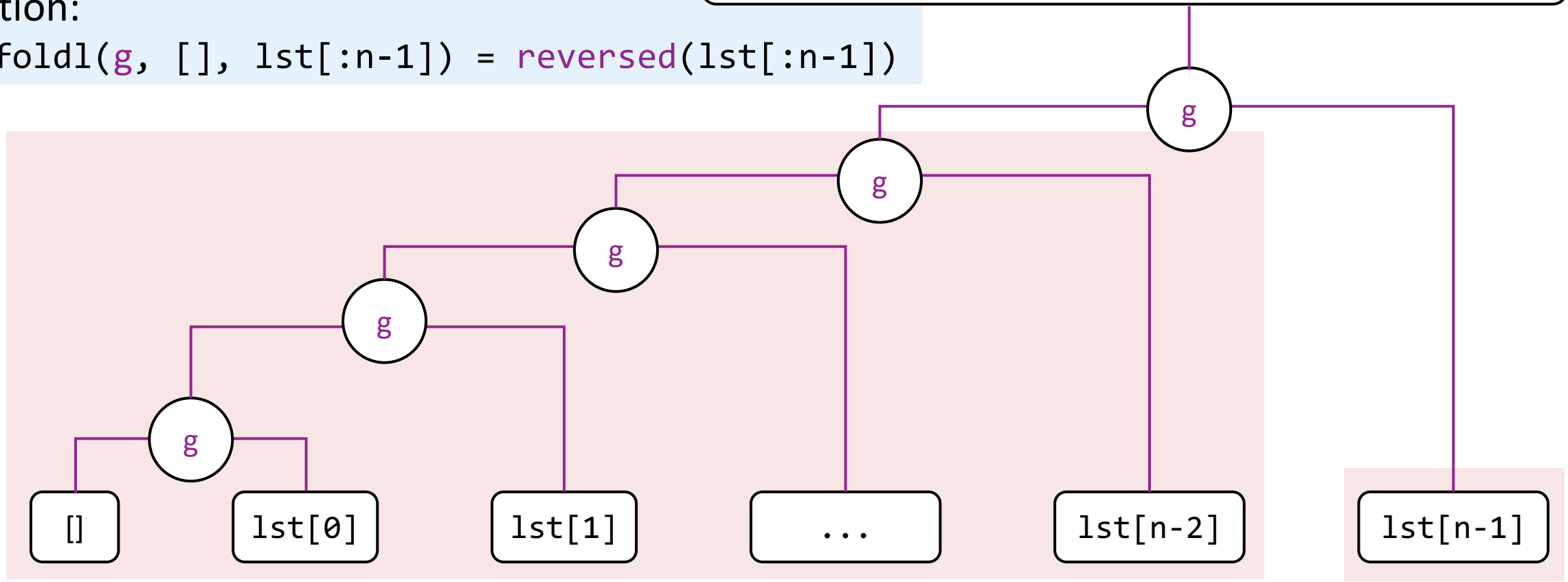
`x = lst[n-1]`

```
def reversed(lst):
    """Returns [lst[n-1], lst[n-2], ..., lst[0]]."""
    return foldl(_____g_____, [], lst)
```

```
g = lambda rlst, x: [x] + rlst
```

Observation:

`rlst = foldl(g, [], lst[:n-1]) = reversed(lst[:n-1])`



`rlst = [lst[n-2], lst[n-3], ..., lst[0]]`

`x = lst[n-1]`

Maybe Not So Many Functions ...

Reimplement Old Friends for Lists

`map(f, lst)` (2 pts)

`filter(p, lst)` (2 pts)

`reversed(lst)` (2 pts)

```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
```

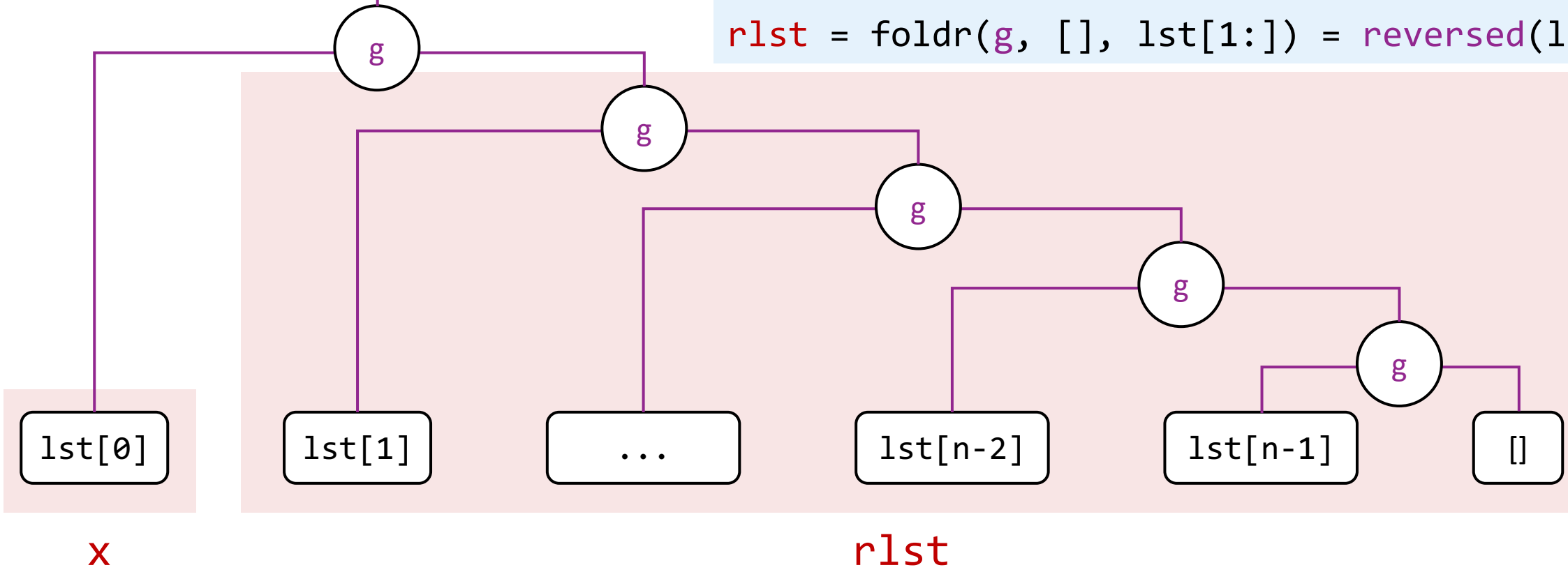
```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
```

```
def reversed(lst):
    """Returns [lst[n-1], lst[n-2], ..., lst[0]]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, rlst: ???
```

Observation:

`rlst = foldr(g, [], lst[1:]) = reversed(lst[1:])`

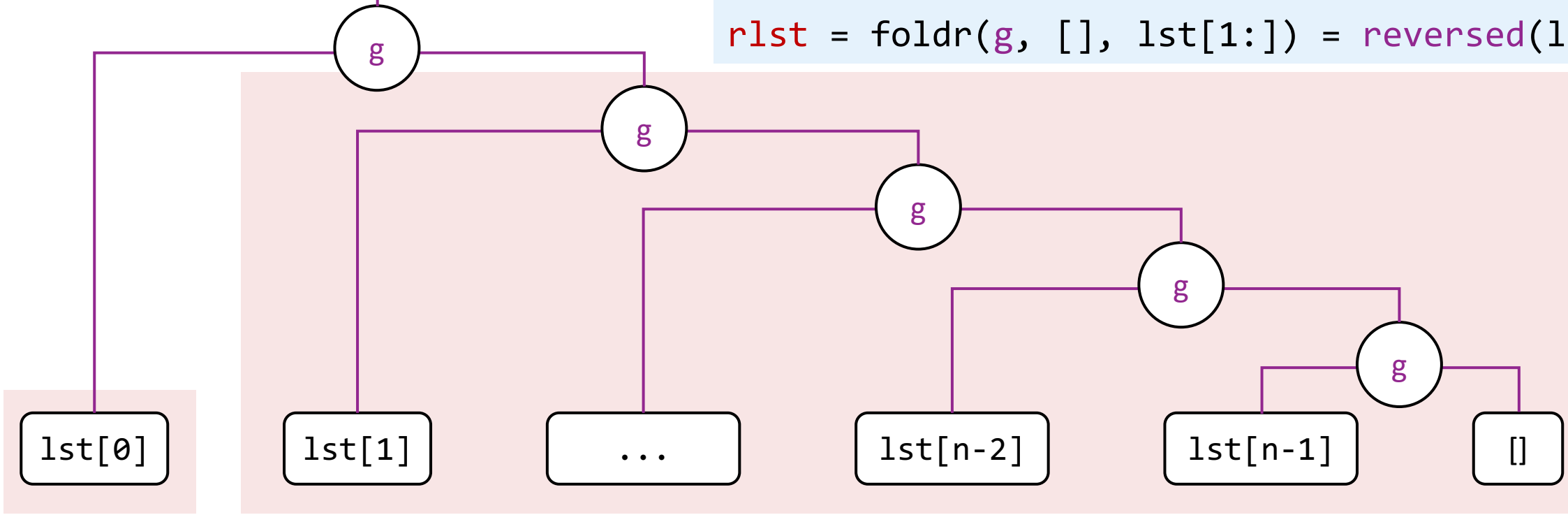


```
def reversed(lst):
    """Returns [lst[n-1], lst[n-2], ..., lst[0]]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, rlst: ???
```

Observation:

`rlst = foldr(g, [], lst[1:]) = reversed(lst[1:])`



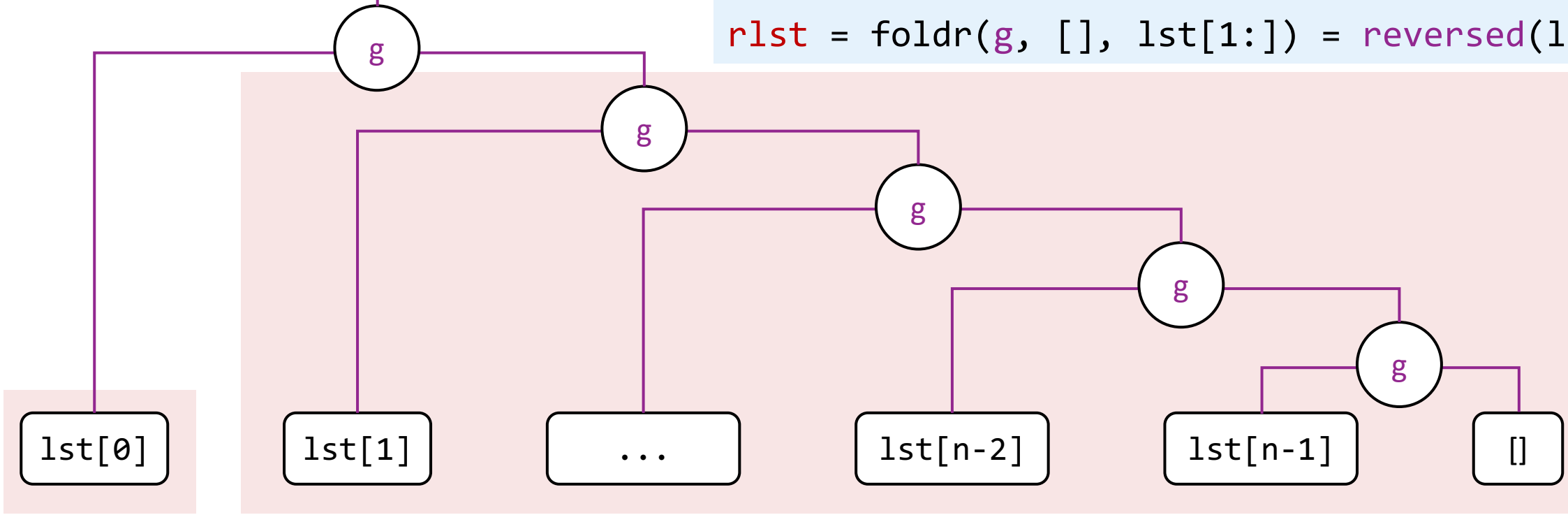
`x = lst[0]` `rlst = [lst[n-1], lst[n-2], ..., lst[1]]`

```
def reversed(lst):
    """Returns [lst[n-1], lst[n-2], ..., lst[0]]."""
    return foldr(_____g_____, [], lst)
```

```
g = lambda x, rlst: rlst + [x]
```

Observation:

`rlst = foldr(g, [], lst[1:]) = reversed(lst[1:])`



`x = lst[0]` `rlst = [lst[n-1], lst[n-2], ..., lst[1]]`

Maybe Not So Many Functions ...

Reimplement Old Friends for Lists

`map(f, lst)` (2 pts)

`filter(p, lst)` (2 pts)

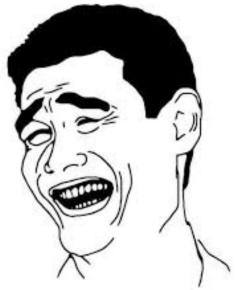
`reversed(lst)` (2 pts)

```
def foldl(f, z, lst):
    """Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
```

```
def foldr(f, z, lst):
    """Returns f(lst[0], f(lst[1], ... f(lst[n-1], z) ...))."""
```

One Function for All !

- Lots of Functions (4 pts)

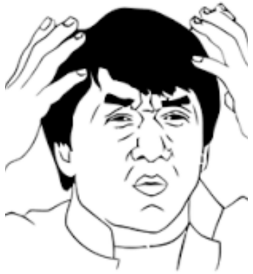


- Maybe Not So Many Functions ... (6 pts)

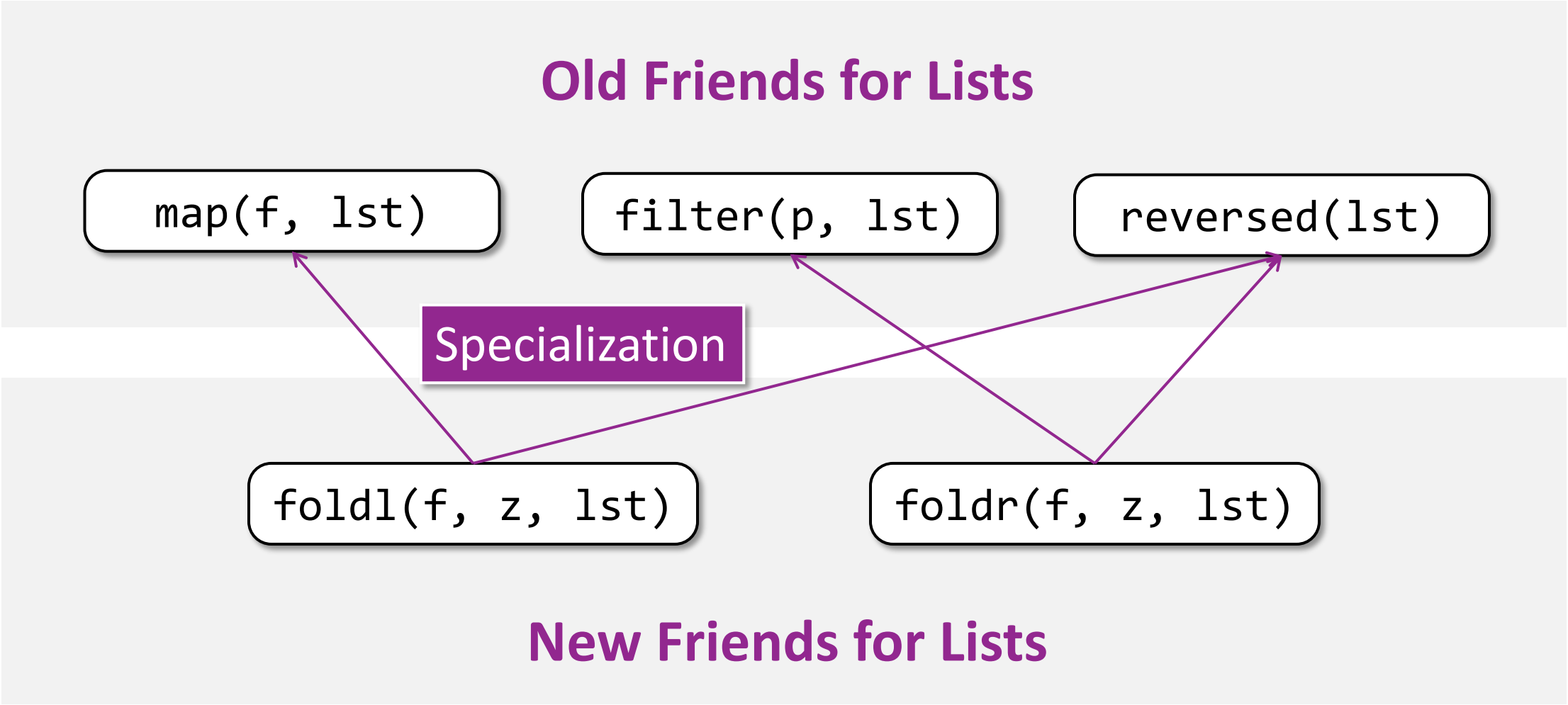
8 - 10pts
Expected



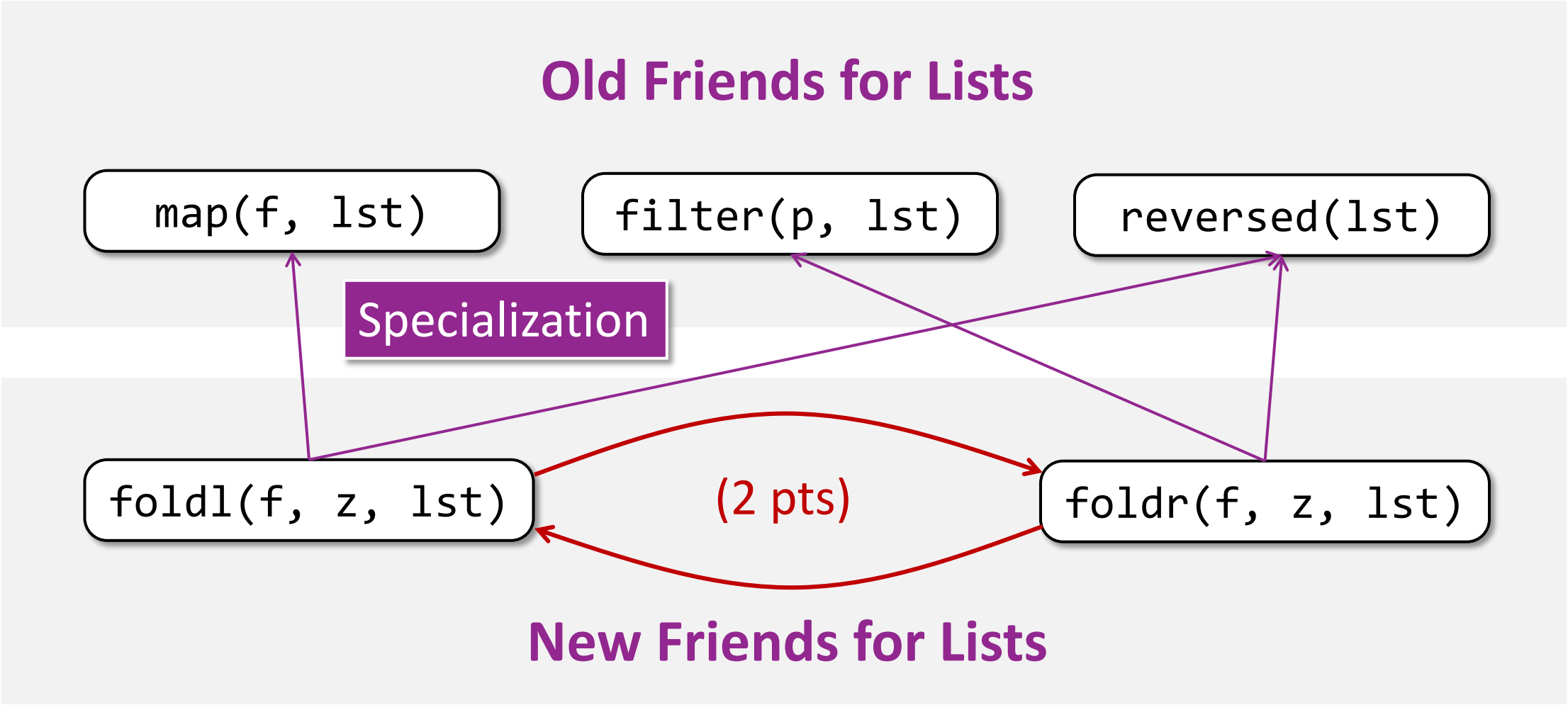
- Just One Function! (2 pts)



Just One Function!



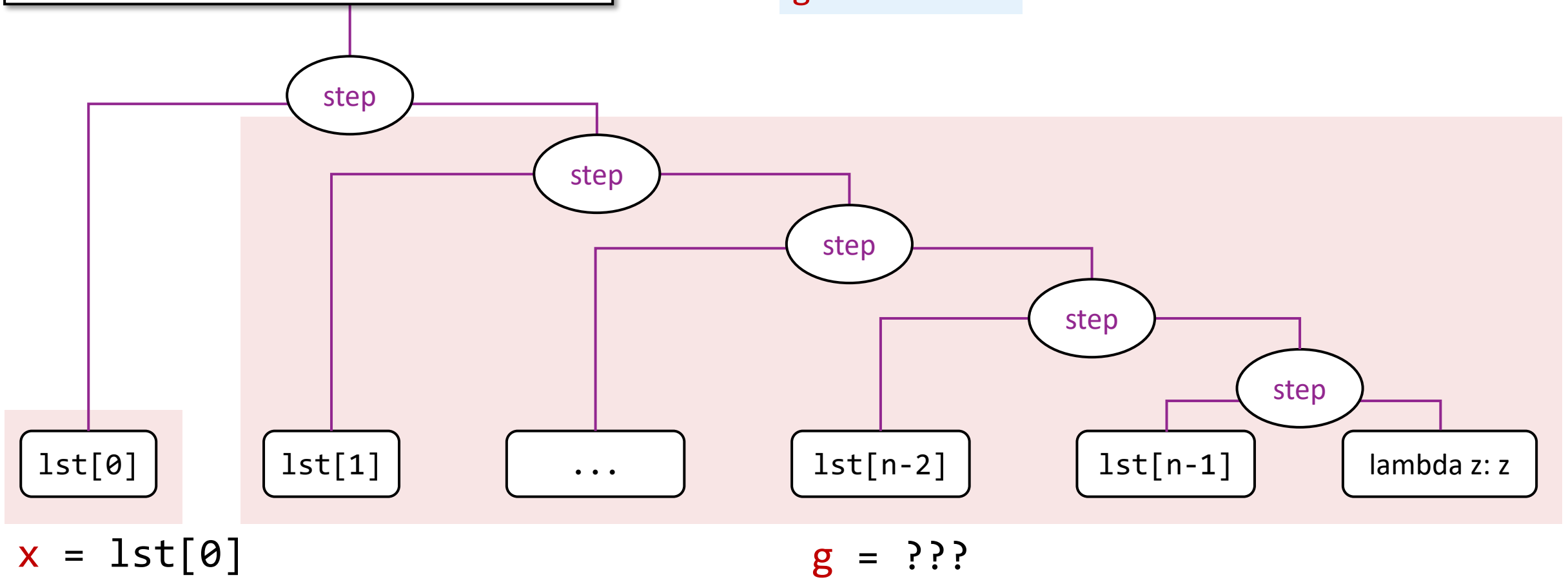
Just One Function!



```
def foldl(f, z, lst):
    """ Returns f(...f(f(z, lst[0]), lst[1])..., lst[n-1])."""
    return foldr(step, lambda z: z, lst)(z)
```

```
def step(x, g):
    return ???
```

Observation:
g = ???



x = lst[0]

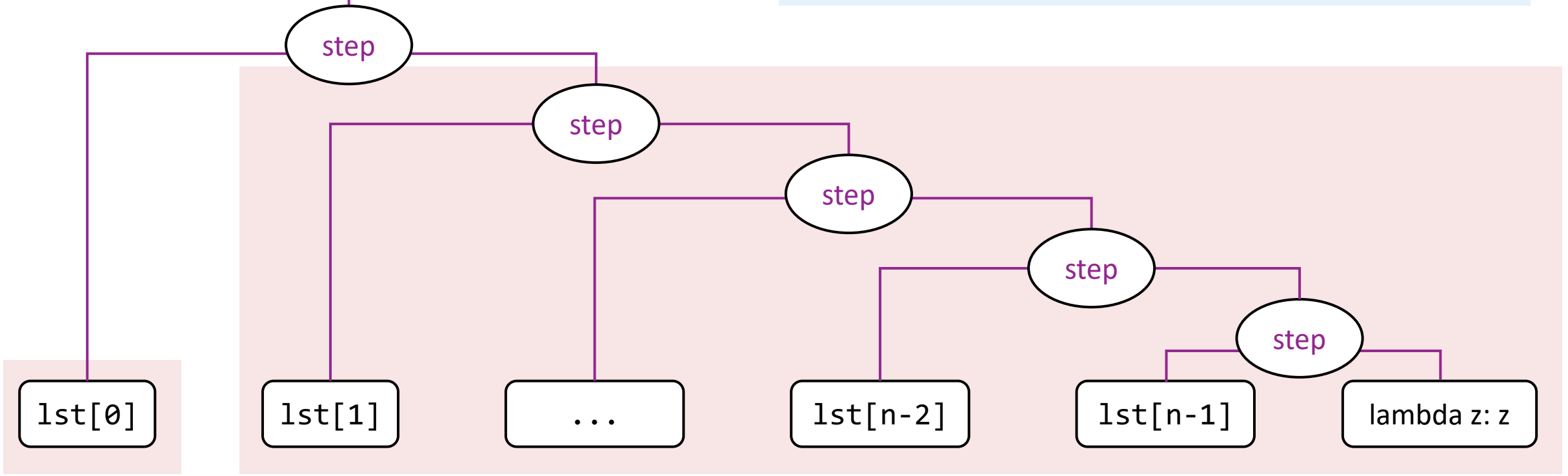
g = ???

```
def foldl(f, z, lst):
    """ Returns (lambda z: f(...f(f(z, lst[0]), lst[1])..., lst[n-1]))(z)."""
    return foldr(step, lambda z: z, lst)(z)
```

```
def step(x, g):
    return lambda z: ???
```

Observation:

```
g = foldr(step, lambda z: z, lst[1:])
  = lambda z: ???
```



`x = lst[0]`

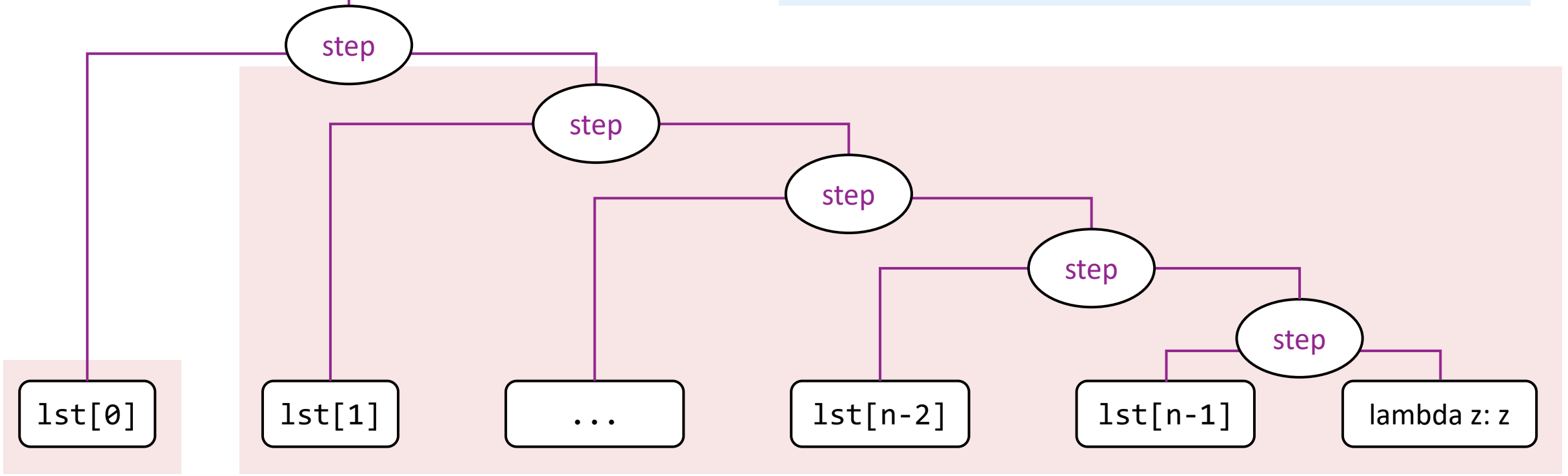
`g = lambda z: ???`

```
def foldl(f, z, lst):
    """ Returns (lambda z: f(...f(f(z, lst[0]), lst[1])..., lst[n-1]))(z)."""
    return foldr(step, lambda z: z, lst)(z)
```

```
def step(x, g):
    return lambda z: g(???)
```

Observation:

```
g = foldr(step, lambda z: z, lst[1:])
  = lambda z: foldl(f, z, lst[1:])
```



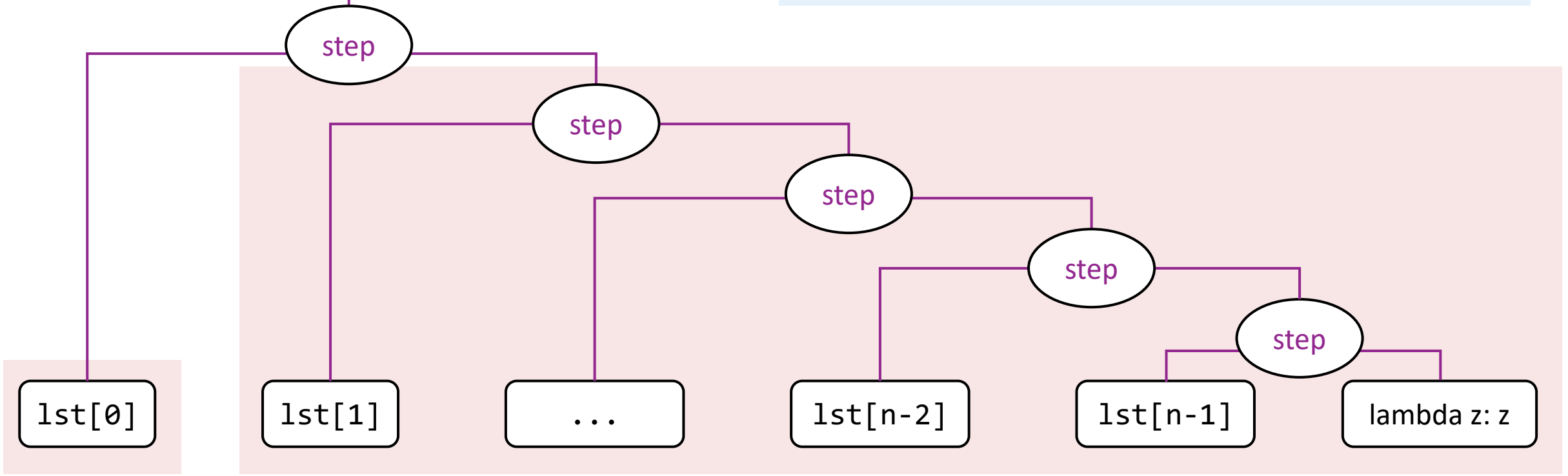

```
x = lst[0]        g = lambda z: f(...f(z, lst[1])..., lst[n-1])
```

```
def foldl(f, z, lst):
    """ Returns (lambda z: f(...f(f(z, lst[0]), lst[1])..., lst[n-1]))(z)."""
    return foldr(step, lambda z: z, lst)(z)
```

```
def step(x, g):
    return lambda z: g(f(z, x))
```

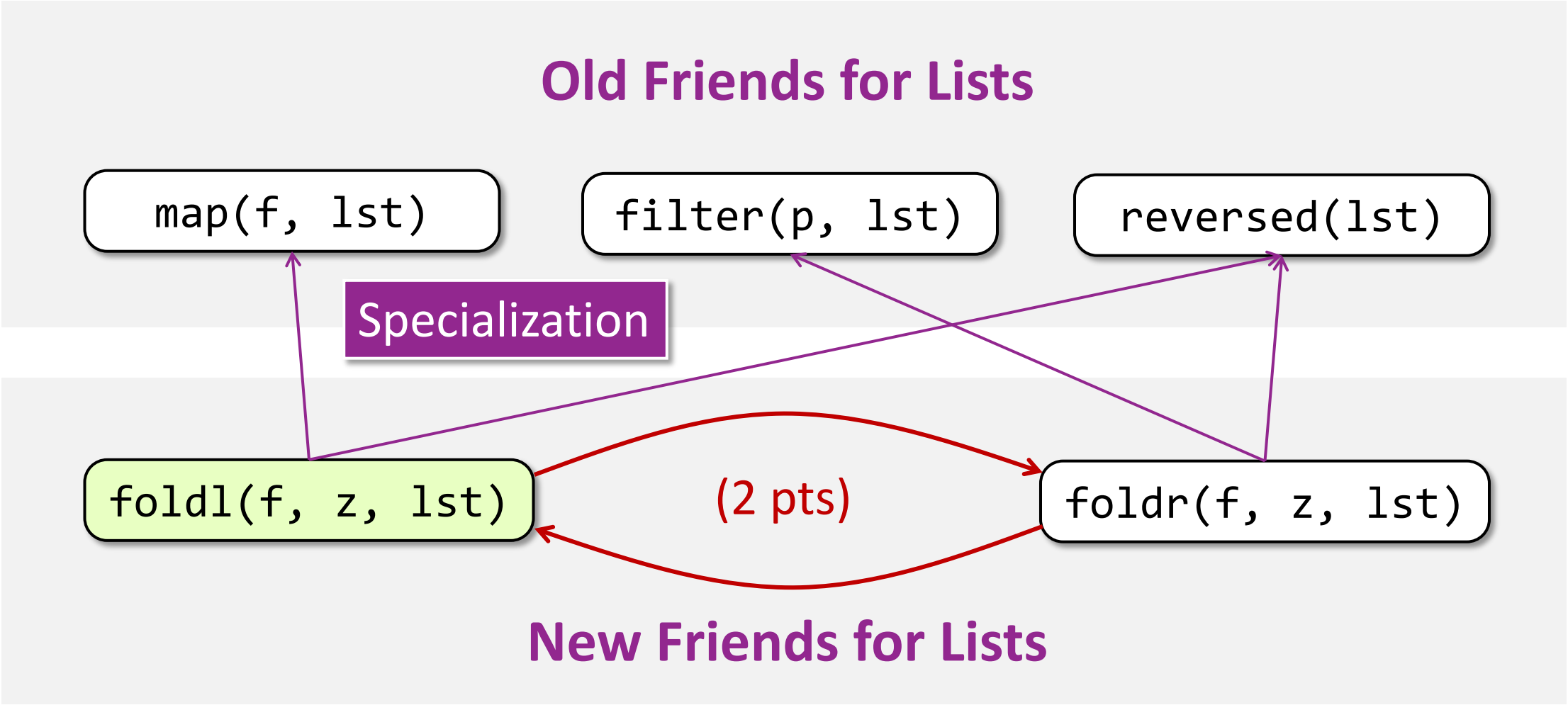
Observation:

```
g = foldr(step, lambda z: z, lst[1:])
  = lambda z: foldl(f, z, lst[1:])
```



`x = lst[0]` `g = lambda z: f(...f(z, lst[1])..., lst[n-1])`

Just One Function!



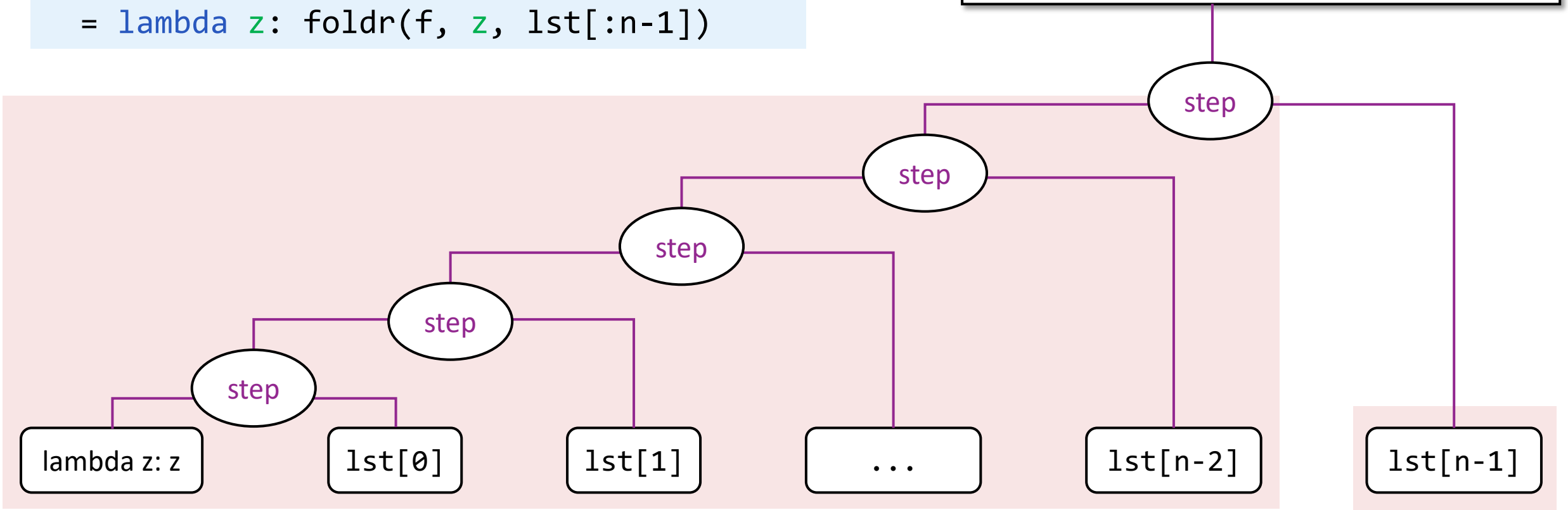
```
def foldr(f, z, lst):
    """ Returns (lambda z: f(lst[0], ... f(lst[n-2], f(lst[n-1], z)) ... ))(z)."""
    return foldl(step, lambda x: x, lst)(z)
```

Observation:

```
g = foldl(step, lambda z: z, lst[:n-1])
  = lambda z: foldr(f, z, lst[:n-1])
```

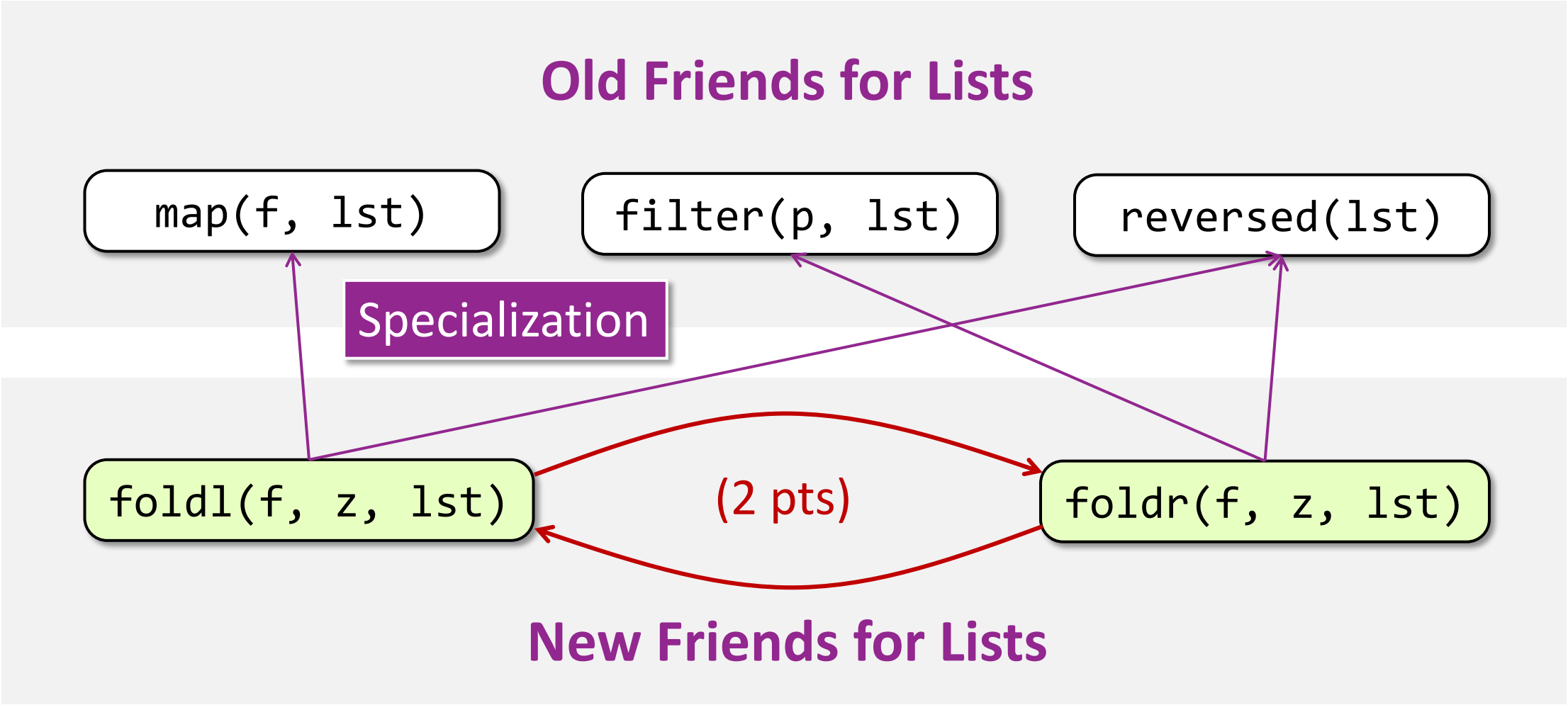
```
def step(g, x):
    return lambda z:
```

Exercise



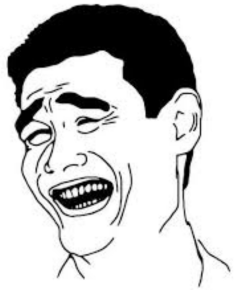
g = lambda z: f(lst[0], ... f(lst[n-2], z) ...) x = lst[n-1]

Just One Function!



One Function for All !

- Lots of Functions (4 pts)

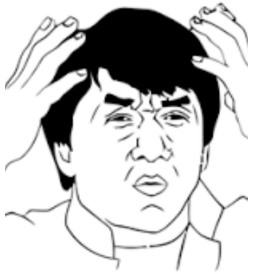


- Maybe Not So Many Functions ... (6 pts)

8 - 10pts
Expected



- Just One Function! (2 pts)



One Function for All !



A TA's Reflection